全栈必备 Docker 入门实战系列教程

主讲教师：（大地）
合作网站：www.itying.com （IT 营）

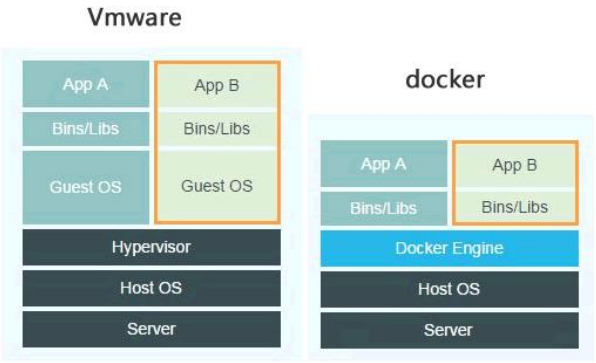
我的专栏：https://www.itying.com/category-79-b0.html

一、Docker 简介与为什么要用 Docker

1.1、Docker 介绍

Docker 是一个跨平台的开源的应用容器引擎，诞生于 2013 年初，基于 Go 语言 并遵从 Apache2.0 协议开源。

刚开始学 **Docker** 你可以把它理解成我们以前学过的虚拟机，但是 Docke 和传统虚拟化方式的不同之处。传统虚拟机技术是虚拟出一套硬件后，在其上运行一个完整操作系统，在该系统上再运行所需应用进程；Docker 相比传统的虚拟化技术要更轻量级，Docker 容器内的应用程序是直接运行在宿主内核中的，容器内没有自己的内核，也没有进行硬件虚拟。



特性	虚拟机	容器
隔离级别	操作系统级	进程级
隔离策略	Hypervisor	CGroups
系统资源	5~15%	0~5%
启动时间	分钟级	秒级
镜像存储	GB-TB	KB-MB
集群规模	上百	上万
高可用策略	备份、容灾、迁移	弹性 负载 动态

因此 Docker 容器要比传统虚拟机占用资源更小、系统支持量更大、启动速度更快、更容易维护和扩展。

目前 **Docker** 是全栈开发者必备的技能之一。

官网：https://hub.docker.com/

1.2、为什么要使用 Docker

除了刚才说的 Docker 容器要比传统虚拟机占用资源更小、系统支持量更大、启动速度更快、更容易维护和扩展外，Docker 还是世界领先的软件容器平台。

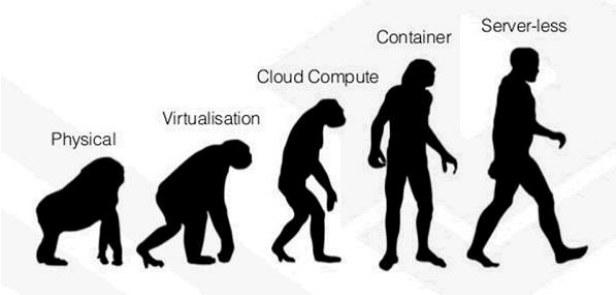
1、开发人员利用 Docker 快速部署 调试我们的应用。

2、开发人员利用 Docker 可以消除协作编码时“在我的机器上可正常工作，其他机器不能正常工作”的问题。Docker 可以提供一致的运行环境，开发过程中一个常见的问题是环境一致性问题。由于开发环境、测试环境、生产环境不一致，导致有些 bug 并未在开发过程中被发现。

3、运维人员利用 Docker 可以在隔离容器中并行运行和管理应用。

4、Serverless 也是基于 docker 容器技术

很多公司在使用 Docker，目前 **Docker** 是全栈开发者必备的技能之一。



1.4、学习 Docker 必备基础

docker 容器都是基于 linux 内核，所以学习 docker 必须具备 linux 基础，如果不会 linux 请参考下面教程。

https://www.itying.com/goods-1035.html

Docker 环境要求

1、如果使用的是 linux 系统安装 docker 需要的最小内核是 3.10

Centos7 和 Centos8 都能满足要求

```
[root@localhost /]# uname -r
4.18.0-305.3.1.el8.x86_64
```

2、Windows 中安装 docker 建议使用 win10

3、macOS must be version 10.14 or newer

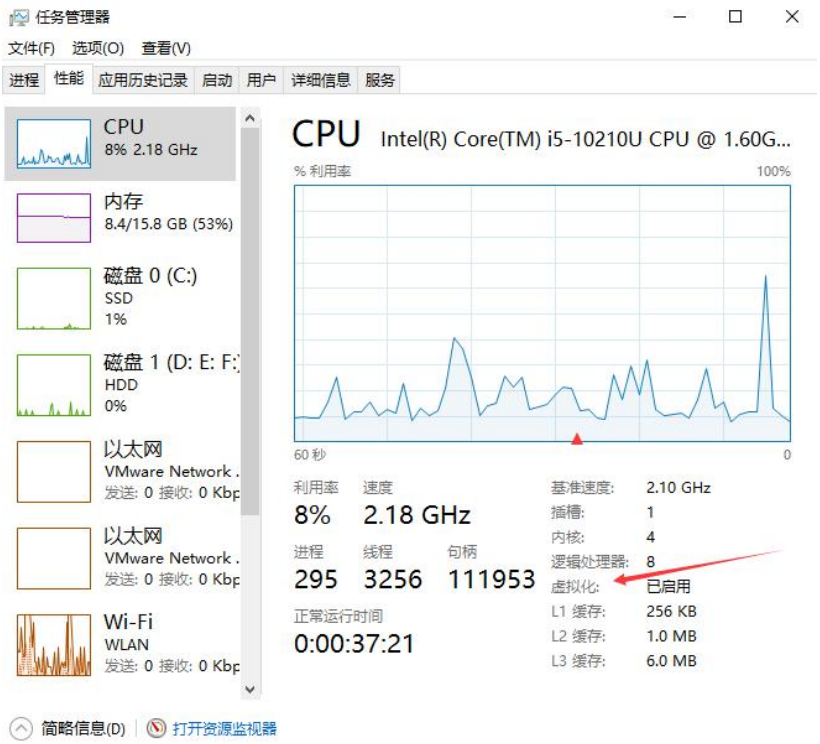
二、Docker 的安装

下面给大家分别演示一下在 Windows macos 以及 linux 中如何安装 docker

2.1、Windows 中安装 Docker

注意：操作系统上面需要启用 Hyper-V 和适用 Linux 的子系统

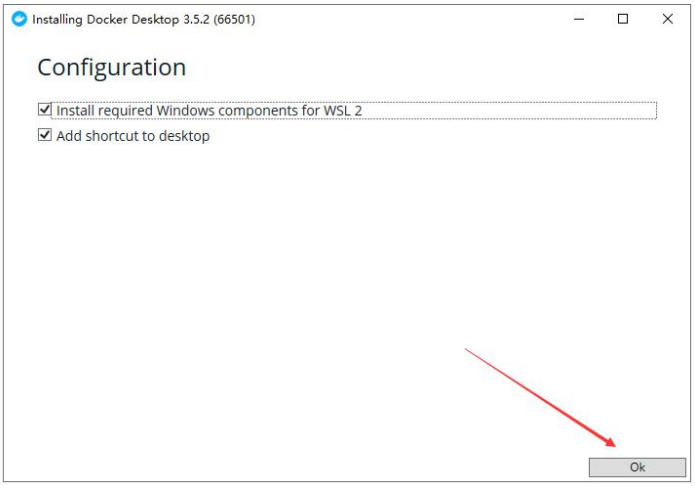
Hyper-V 是微软开发的虚拟机，类似于 VMWare 或 VirtualBox，仅适用于 Windows 10。这是 Docker Desktop for Windows 所使用的虚拟机。但是，这个虚拟机一旦启用，QEMU、VirtualBox 或 VMWare Workstation 15 及以下版本将无法使用！如果你必须在电脑上使用其他虚拟机（例如开发 Android 应用必须使用的模拟器），请不要使用 Hyper-V！

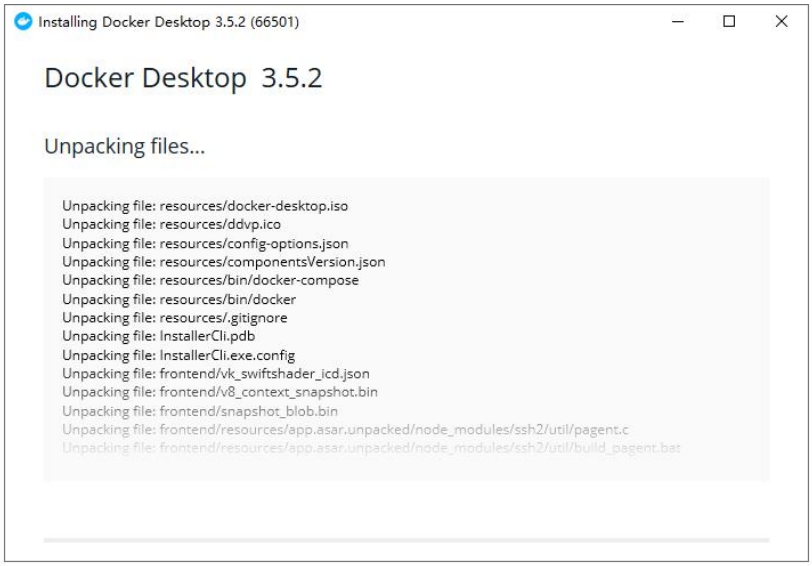


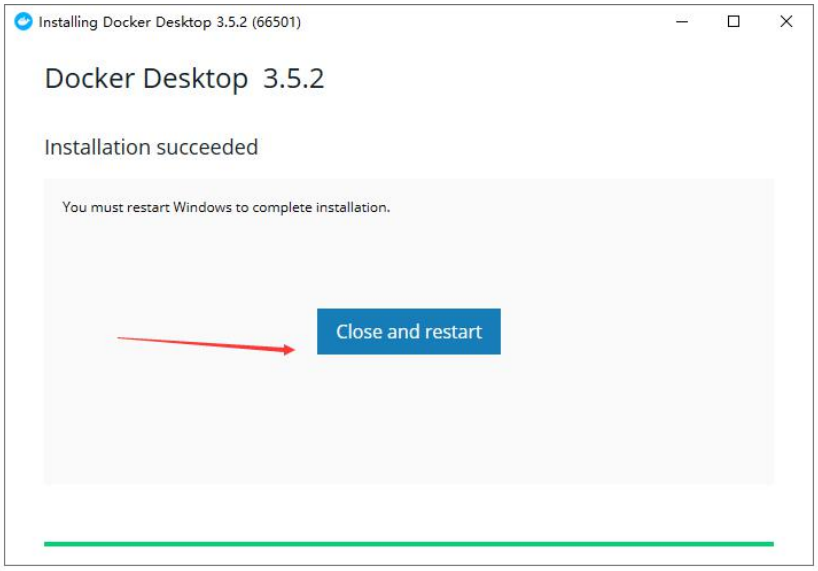
2.1.1、下载软件包

下载软件包：https://docs.docker.com/engine/install/

2.1.2、安装软件



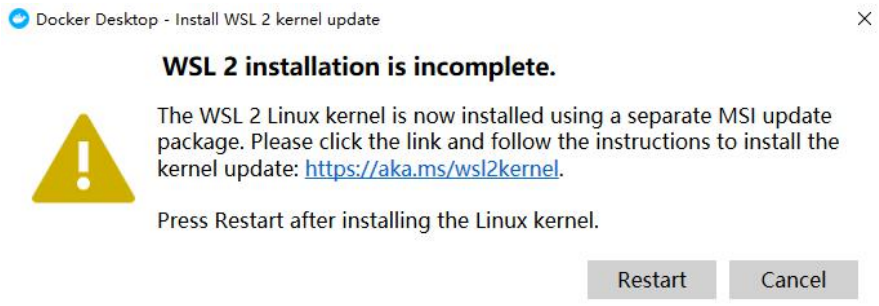




注意：此方法仅适用于 Windows 10 操作系统专业版、企业版、教育版和部分家庭版！

2.13、运行软件

如果第一次打开提示下面错误，请下载安装 wsl*update*x64.mis 后重启 docker



https://docs.microsoft.com/en-us/windows/wsl/install-win10#step-4---download-the-linux-kernel-update-package

下载 wsl*update*x64.mis 安装后重启 docker

```
docker info
```

```
C:\Users\htzhanglong>docker info
Client:
 Context:    default
 Debug Mode: false
 Plugins:
  buildx: Build with BuildKit (Docker Inc., v0.5.1-docker)
  compose: Docker Compose (Docker Inc., v2.0.0-beta.6)
  scan: Docker Scan (Docker Inc., v0.8.0)

Server:
 Containers: 0
  Running: 0
  Paused: 0
  Stopped: 0
 Images: 0
 Server Version: 20.10.7
 Storage Driver: overlay2
  Backing Filesystem: extfs
  Supports d_type: true
  Native Overlay Diff: true
  userxattr: false
 Logging Driver: json-file
 Cgroup Driver: cgroupfs
 Cgroup Version: 1
 Plugins:
```

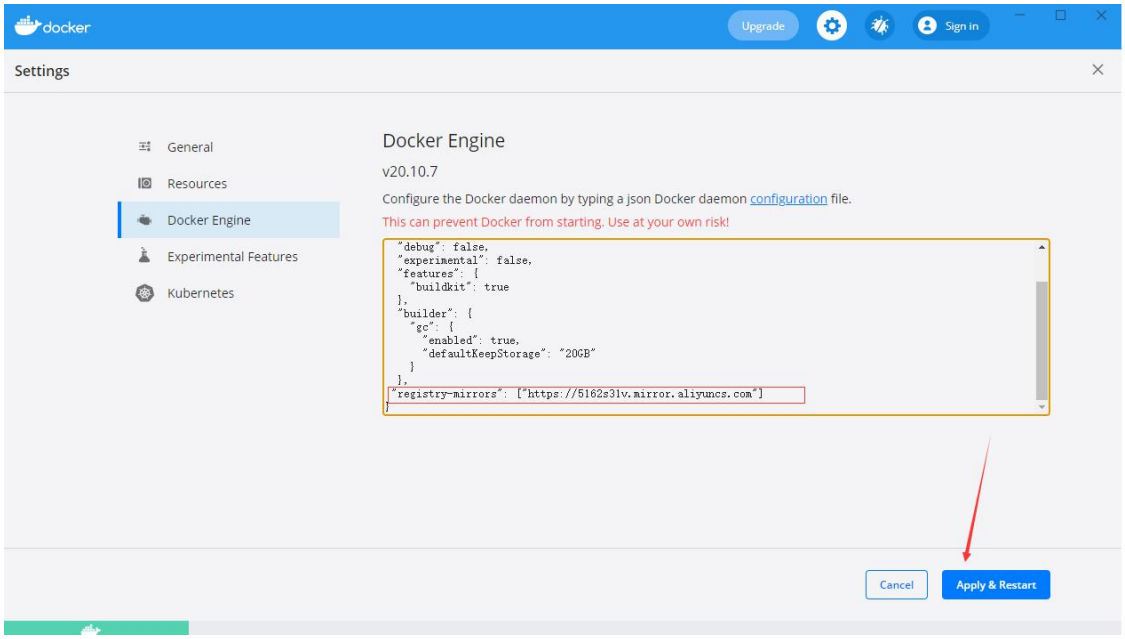
2.1.4、镜像加速

鉴于国内网络问题，后续拉取 Docker 镜像十分缓慢，我们可以需要配置加速器来解决，我使用的是阿里的镜像地址：https://02xz0m84.mirror.aliyuncs.com

如何创建阿里镜像参考：http://bbs.itying.com/topic/6103bb600ebf8e02588af899

在任务栏点击 Docker for mac 应用图标 -> Perferences... -> Docker Engine

```
{
  ...
  "registry-mirrors": ["https://02xz0m84.mirror.aliyuncs.com"]
  ...
}
```



2.1.5、通过运行 hello world 映像来验证 Docker 引擎安装是否正确

启动 hello-world 容器

```
docker run hello-world
[root@localhost /]# docker run hello-world

Unable to find image 'hello-world:latest' locally

latest: Pulling from library/hello-world

b8dfde127a29: Pull complete

Digest: sha256:df5f5184104426b65967e016ff2ac0bfcd44ad7899ca3bbcf8e44e4461491a9e
```

```
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
```

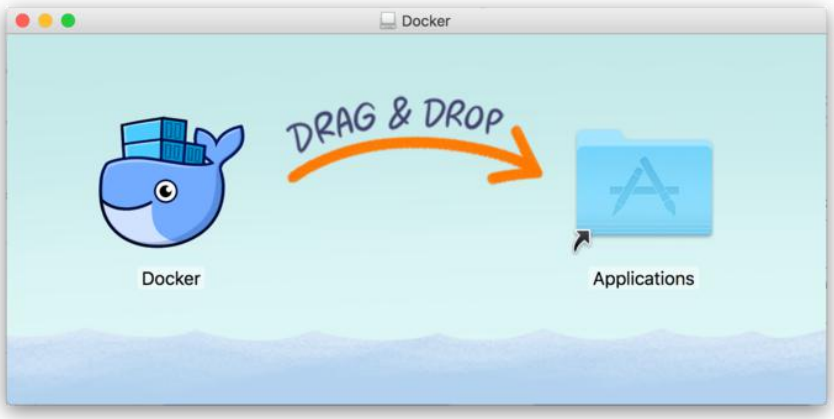
https://docs.docker.com/get-started/

2.2、Macos 中安装 Docker

2.2.1、下载安装

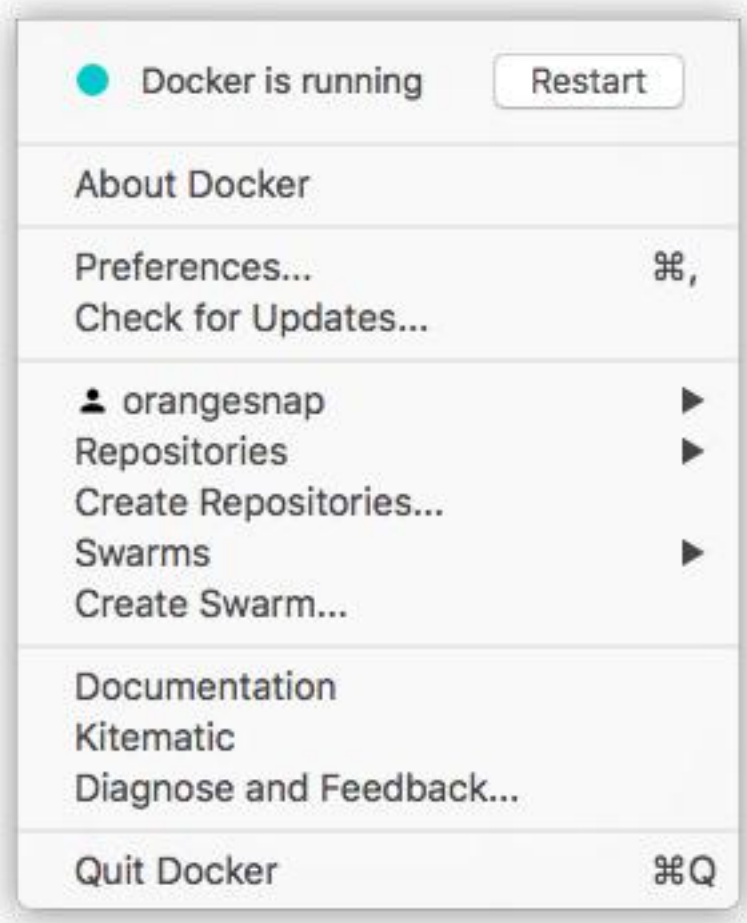
https://docs.docker.com/docker-for-mac/install/

如同 macOS 其它软件一样，安装也非常简单，双击下载的 .dmg 文件，然后将鲸鱼图标拖拽到 Application 文件夹即可。

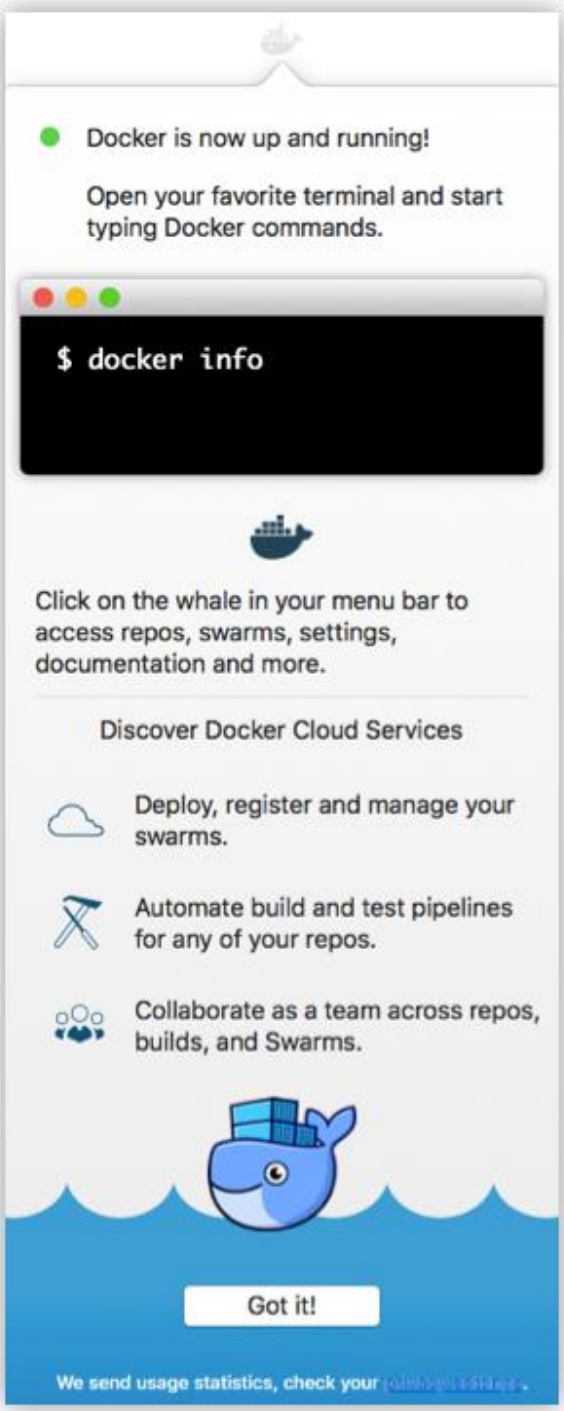


从应用中找到 Docker 图标并点击运行。可能会询问 macOS 的登陆密码，输入即可。

点击顶部状态栏中的鲸鱼图标会弹出操作菜单。



第一次点击图标，可能会看到这个安装成功的界面，点击 "Got it!" 可以关闭这个窗口。



启动终端后，通过命令可以检查安装后的 Docker 版本。

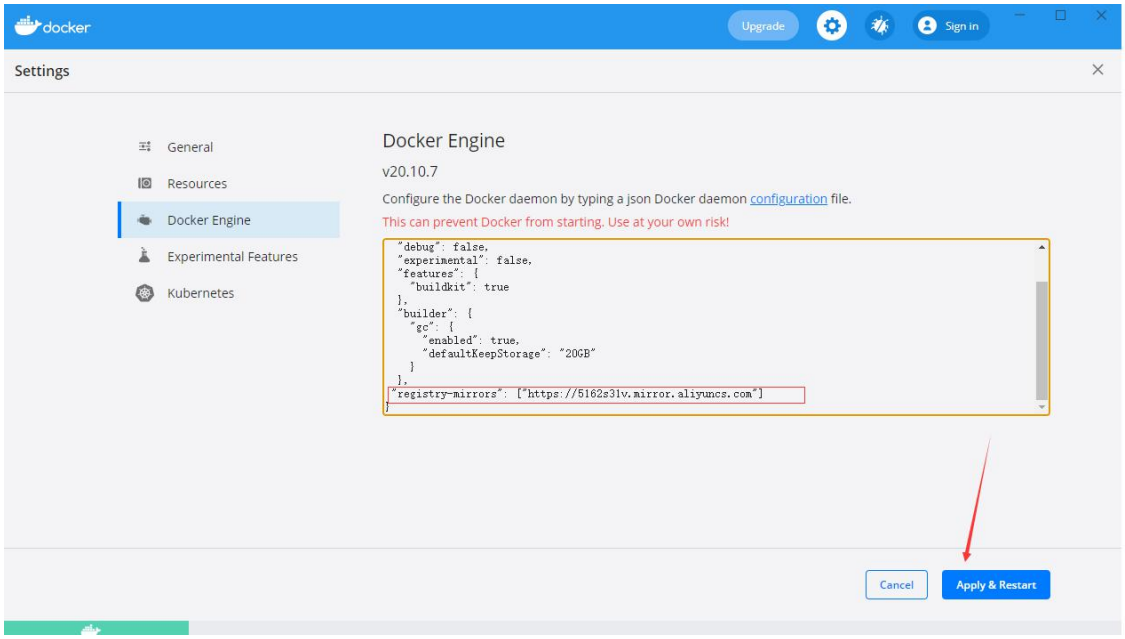
```
$ docker --version
Docker version 17.09.1-ce, build 19e2cf6
```

2.2.2、镜像加速

鉴于国内网络问题，后续拉取 Docker 镜像十分缓慢，我们可以需要配置加速器来解决，我使用的是阿里云的镜像地址：https://02xz0m84.mirror.aliyuncs.com

在任务栏点击 Docker for mac 应用图标 -> Perferences... -> Docker Engine

```
{
  ...
  "registry-mirrors": ["https://02xz0m84.mirror.aliyuncs.com"]
  ...
}
```



之后我们可以通过 docker info 来查看是否配置成功。

```
$ docker info
...
Registry Mirrors:
 http://hub-mirror.c.163.com
Live Restore Enabled: false
```

2.2.3、通过运行 hello world 映像来验证 Docker 引擎安装是否正确

启动 hello-world 容器

```
docker run hello-world
[root@localhost /]# docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
b8dfde127a29: Pull complete
Digest: sha256:df5f5184104426b65967e016ff2ac0bfcd44ad7899ca3bbcf8e44e4461491a9e
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
```

```
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

2.3、Linux 中安装 docker

建议使用 Centos7 或者 Centos8，Centos7 和 Centos8 用法配置都是一样的。

linux 安装 docker 官方文档：https://docs.docker.com/install/linux/docker-ce/centos/

2.3.1、Linux 中安装 Docker 的准备工作

为了方便测试请关闭 selinux、关闭防火墙。

SELinux 防火墙的设置：

```
[root@localhost ~]# getenforce
Disabled
修改/etc/selinux/config 文件
将 SELINUX=enforcing 改为 SELINUX=disabled
```

Firewalld 防火墙的设置：

```
1、firewalld 的基本使用:
    启动： systemctl start firewalld
    关闭： systemctl stop firewalld
    查看状态： systemctl status firewalld
    开机禁用 ： systemctl disable firewalld
    开机启用 ： systemctl enable firewalld

2、firewall-cmd 的基本使用:
    那怎么开启一个端口呢:
        firewall-cmd --zone=public --add-port=80/tcp --permanent （–permanent 永久生效，没有此参数重启后失效）
    重新载入:
        firewall-cmd --reload          修改 firewall-cmd 配置后必须重启
    查看:
        firewall-cmd --zone= public --query-port=80/tcp
    删除:
        firewall-cmd --zone= public --remove-port=80/tcp --permanent

    查看所有打开的端口：
        firewall-cmd --zone=public --list-ports
```

2.3.2、Linux Centos 中安装 Docker

安装需要的软件包：

```
yum install -y yum-utils
```

配置 docker 源

```
yum-config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
```

或者阿里云源

```
yum-config-manager --add-repo http://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo
```

```
yum search docker
```

安装 docker

```
yum install docker-ce docker-ce-cli containerd.io -y
```

- containerd.io - daemon to interface with the OS API (in this case, LXC - Linux Containers), essentially decouples Docker from the OS, also provides container services for non-Docker container managers
- docker-ce - Docker daemon, this is the part that does all the management work, requires the other two on Linux
- docker-ce-cli - CLI tools to control the daemon, you can install them on their own if you want to control a remote Docker daemon

2.3.3 、启动 docker

```
systemctl start docker
```

开机启动

```
systemctl enable docker
```

看 docker 状态

```
systemctl status docker
```

查看自启动

```
systemctl list-unit-files|grep enabled
systemctl list-unit-files | grep enabled |grep docker
```

```
[root@localhost /]# systemctl list-unit-files | grep enabled |grep docker
docker.service                              enabled
```

第一个 docker 命令

```
docker info
```

```
[root@localhost /]# docker info
Client:
 Context:    default
 Debug Mode: false
 Plugins:
  app: Docker App (Docker Inc., v0.9.1-beta3)
  buildx: Build with BuildKit (Docker Inc., v0.5.1-docker)
  scan: Docker Scan (Docker Inc., v0.8.0)

Server:
 Containers: 0
  Running: 0
  Paused: 0
  Stopped: 0
 Images: 0
 Server Version: 20.10.7
 Storage Driver: overlay2
```

第二个命令查看 docker 版本

```
docker --version
```

2.3.4、安装指定版本的 docker

要安装*特定版本*的 Docker Engine，请在 repo 中列出可用版本，然后选择并安装：

一种。列出并排序您的存储库中可用的版本。此示例按版本号对结果进行排序，从高到低，并被截断：

```
yum list docker-ce --showduplicates | sort -r
```

```
sudo yum install docker-ce-<VERSION_STRING> docker-ce-cli-<VERSION_STRING> containerd.io
```

2.3.5、docker daemon.json 配置阿里云加速器

当我们需要对 docker 服务进行调整配置时，不用去修改主文件 docker.service 的参数，通过 daemon.json 配置文件来管理，更为安全、合理。对于

```
mkdir -p /etc/docker
vi /etc/docker/daemon.json
{
  "registry-mirrors": ["https://02xz0m84.mirror.aliyuncs.com"]
}
```

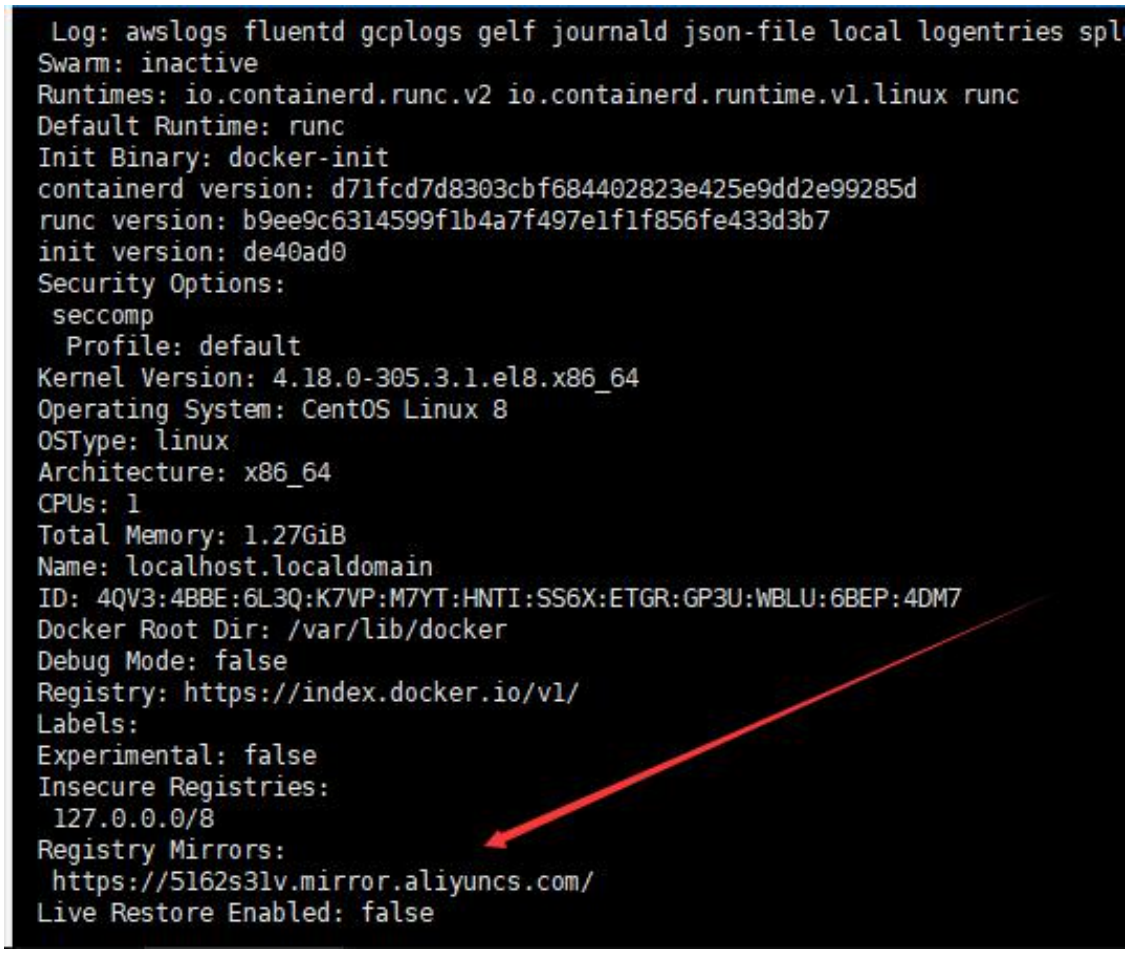
如果没有/etc/docker 这个目录就创建这个目录:

```
[root@localhost /]# vi /etc/docker/daemon.json
[root@localhost /]# cat /etc/docker/daemon.json
{
  "registry-mirrors": ["https://5162s31v.mirror.aliyuncs.com"]
}
[root@localhost /]#
```

重新加载 daemon 重启 docker

加载配置文件

```
systemctl daemon-reload
systemctl restart docker
docker info
```



2.3.6、通过运行 hello world 映像来验证 Docker 引擎安装是否正确

启动 hello-world 容器

```
docker run hello-world
[root@localhost /]# docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
b8dfde127a29: Pull complete
Digest: sha256:df5f5184104426b65967e016ff2ac0bfcd44ad7899ca3bbcf8e44e4461491a9e
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

```
To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

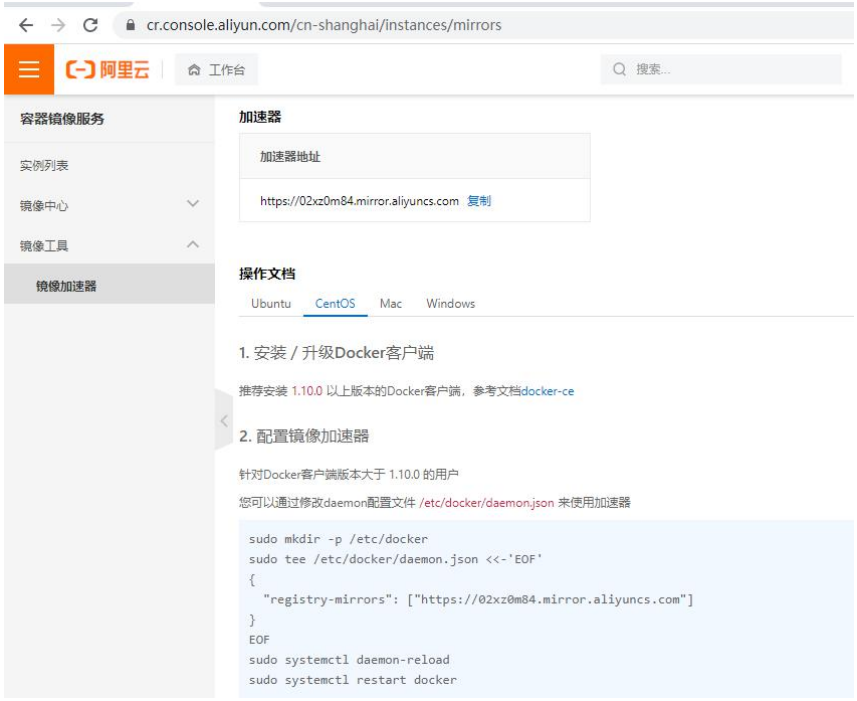
2.3.7、卸载 docker

1. 卸载 Docker Engine、CLI 和 Containerd 包：
   $ sudo yum remove docker-ce docker-ce-cli containerd.io
2. 主机上的映像、容器、卷或自定义配置文件不会自动删除。删除所有镜像、容器和卷：
   $ sudo rm -rf /var/lib/docker
   $ sudo rm -rf /var/lib/containerd

您必须手动删除任何已编辑的配置文件。

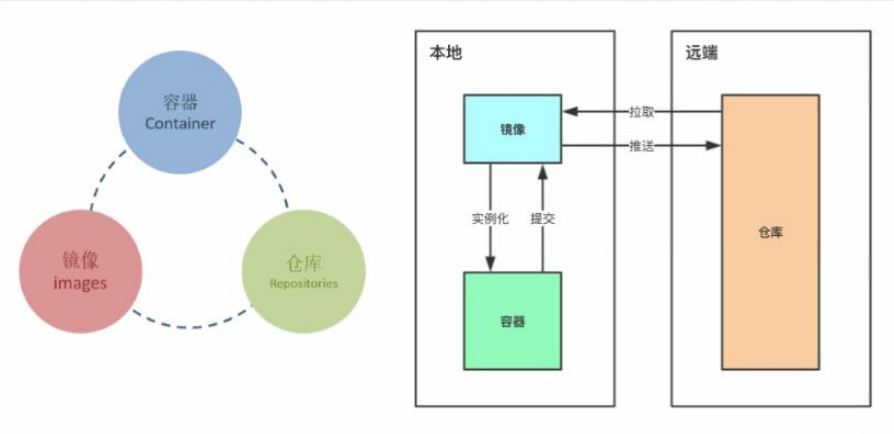
2.4、阿里云 Docker 镜像加速器

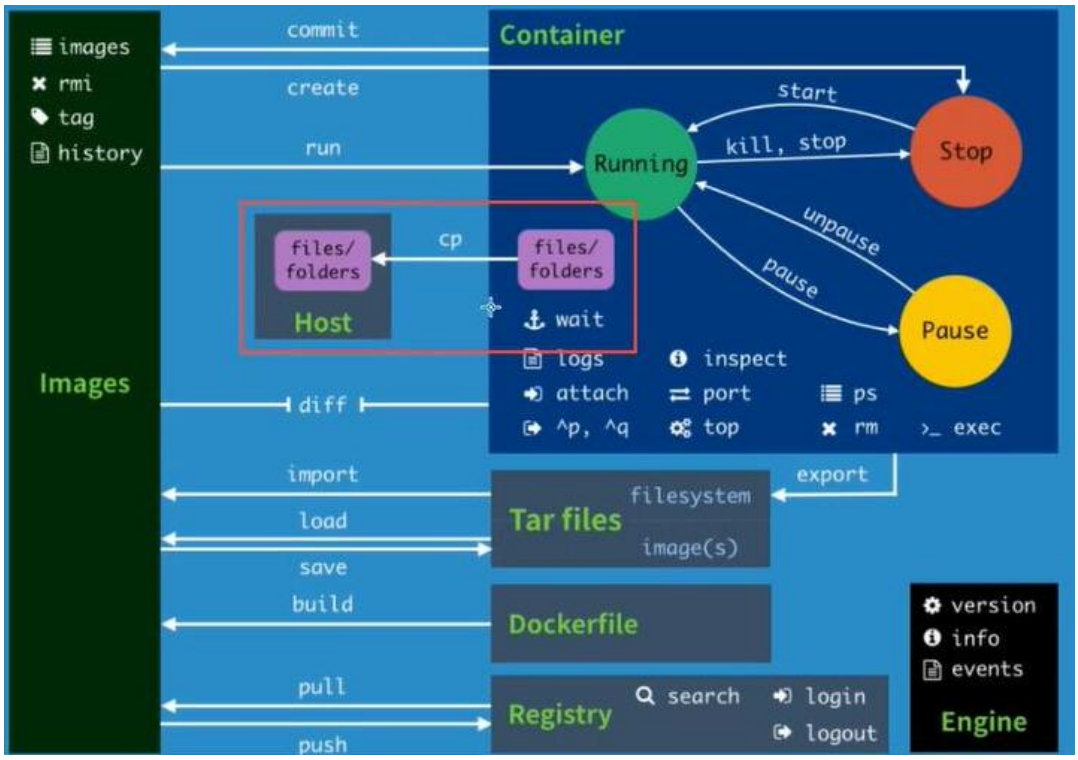
访问 https://www.aliyun.com/ 搜索“容器镜像服务”



三、Docker 镜像 容器 仓库

3.1、Docker 镜像 容器 仓库的简单介绍



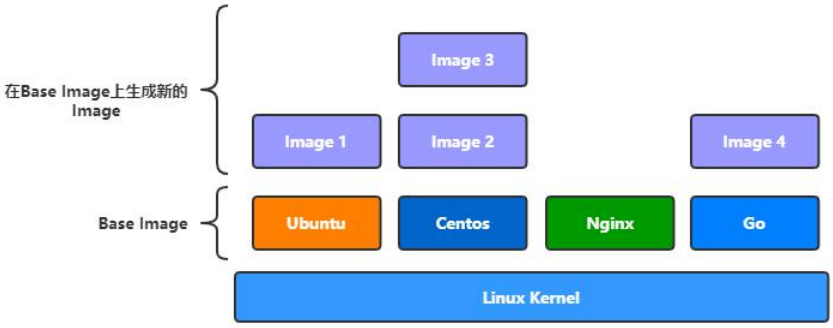


3.1.1、镜像

Docker 镜像就是一个 Linux 的文件系统（Root FileSystem），这个文件系统里面包含可以运行在 Linux 内核的程序以及相应的数据。

这里要强调一下镜像的两个特征：

1. 镜像是分层（Layer）的：即一个镜像可以多个中间层组成，多个镜像可以共享同一中间层，我们也可以通过在镜像添加多一层来生成一个新的镜像。
2. 镜像是只读的（read-only）：镜像在构建完成之后，便不可以再修改，而上面我们所说的添加一层构建新的镜像，这中间实际是通过创建一个临时的容器，在容器上增加或删除文件，从而形成新的镜像，因为容器是可以动态改变的。

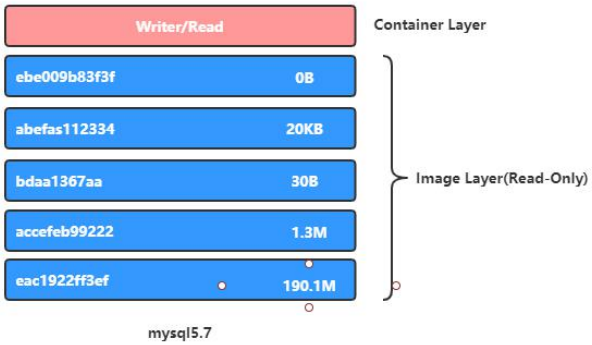


3.1.2、容器

类似 linux 系统环境，运行和隔离应用。容器从镜像启动的时候，docker 会在镜像的最上一层创建一个可写层，镜像本身是只读的，保持不变。容器与镜像的关系，就如同面向编程中对象与类之间的关系。

因为容器是通过镜像来创建的，所以必须先有镜像才能创建容器，而生成的容器是一个独立于宿主机的隔离进程，并且有属于容器自己的网络和命名空间。

我们前面介绍过，镜像由多个中间层（layer）组成，生成的镜像是只读的，但容器却是可读可写的，这是因为容器是在镜像上面添一层读写层（writer/read layer）来实现的，如下图所示：



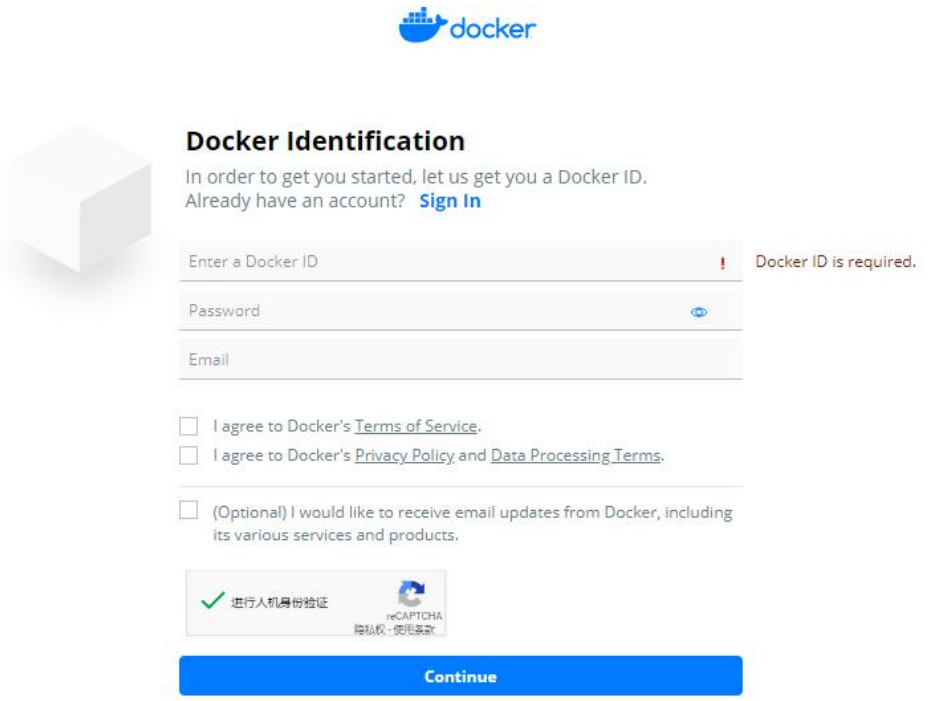
3.1.3、仓库（Repository）

仓库（Repository）是集中存储镜像的地方，这里有个概念要区分一下，那就是仓库与仓库服务器(Registry)是两回事，像我们上面说的 Docker Hub，就是 Docker 官方提供的一个仓库服务器，不过其实有时候我们不太需要太过区分这两个概念。

公共仓库

公共仓库一般是指 Docker Hub，前面我们已经多次介绍如何从 Docker Hub 获取镜像，除了获取镜像外，我们也可以将自己构建的镜像存放到 Docker Hub，这样，别人也可以使用我们构建的镜像。

不过要将镜像上传到 Docker Hub，必须先在 Docker 的官方网站上注册一个账号，注册界面如下，按要求填写必要的信息就可以注册了，很简单的。



私有仓库

有时候自己部门内部有一些镜像要共享时，如果直接导出镜像拿给别人又比较麻烦，使用像 Docker Hub 这样的公共仓库又不是很方便，这时候我们可以自己搭建属于自己的私有仓库服务，用于存储和分布我们的镜像。

3.2、Docker 镜像以及仓库

Docker 镜像就是一个 Linux 的文件系统（Root FileSystem），这个文件系统里面包含可以运行在 Linux 内核的程序以及相应的数据。

3.2.1、docker search 搜索镜像

```
docker search centos
```


```
[root@localhost /]# docker search --help

Usage:  docker search [OPTIONS] TERM

Search the Docker Hub for images

Options:
  -f, --filter filter   Filter output based on conditions provided
      --format string   Pretty-print search using a Go template
      --limit int       Max number of search results (default 25)
      --no-trunc        Don't truncate output


docker search mysql
```

备注：-f STARS=3000 搜索 STARS 大于 3000 的镜像

```
[root@localhost /]# docker search mysql -f STARS=3000
NAME        DESCRIPTION                                     STARS     OFFICIAL
 AUTOMATED
mysql       MySQL is a widely used, open-source relation…   11171     [OK]
mariadb     MariaDB Server is a high performing open sou…   4242      [OK]
```

3.2.2、 docker pull 下载镜像

```
docker pull centos
```

```
[root@localhost /]# docker pull centos
Using default tag: latest
latest: Pulling from library/centos
7a0437f04f83: Pull complete
Digest: sha256:5528e8b1b1719d34604c87e11dcd1c0a20bedf46e83b5632cdeac91b8c04efc1
Status: Downloaded newer image for centos:latest
docker.io/library/centos:latest
```

下载指定的 tag：

```
docker pull centos:8.3.2011
[root@localhost /]# docker pull centos:8.3.2011
8.3.2011: Pulling from library/centos
Digest: sha256:5528e8b1b1719d34604c87e11dcd1c0a20bedf46e83b5632cdeac91b8c04efc1
Status: Downloaded newer image for centos:8.3.2011
docker.io/library/centos:8.3.2011
```

镜像结构: registry*name/repository*name/image*name:tag*name

例如：docker.io/library/centos:8.3.2011

3.1.4、docker images 查看本地镜像

```
docker images
[root@localhost /]# docker images
REPOSITORY      TAG          IMAGE ID          CREATED           SIZE
hello-world     latest       d1165f221234      4 months ago      13.3kB
centos          latest       300e315adb2f      7 months ago      209MB
```

3.1.5、docker tag 给镜像打标签

给镜像打标签可以创建自己的镜像

镜像结构: registry*name/repository*name/image*name:tag*name

例如：docker.io/library/centos:8.3.2011

基本语法：

```
[root@localhost /]# docker tag --help

Usage:  docker tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]

Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE
[root@localhost /]# docker images
REPOSITORY      TAG             IMAGE ID        CREATED          SIZE
hello-world     latest       d1165f221234    4 months ago     13.3kB
centos          8.3.2011     300e315adb2f    7 months ago     209MB
centos          latest       300e315adb2f    7 months ago     209MB
```

给 IMAGE ID 为 300e315adb2f 的镜像打个标签

```
docker tag 300e315adb2f docker.io/itying/centos:v1.0.1
```

```
[root@localhost /]# docker tag 300e315adb2f docker.io/itying/centos:v1.0.1
[root@localhost /]# docker images
REPOSITORY         TAG             IMAGE ID        CREATED          SIZE
hello-world        latest       d1165f221234    4 months ago     13.3kB
itying/centos      v1.0.1       300e315adb2f    7 months ago     209MB
centos             8.3.2011     300e315adb2f    7 months ago     209MB
centos             latest       300e315adb2f    7 months ago     209MB
```

给 IMAGE ID 为 300e315adb2f 的镜像重新打个 latest 标签

```
[root@localhost /]# docker tag 300e315adb2f docker.io/itying/centos:latest
[root@localhost /]# docker images
```

```
REPOSITORY          TAG              IMAGE ID          CREATED           SIZE
hello-world         latest           d1165f221234      4 months ago      13.3kB
centos              8.3.2011         300e315adb2f      7 months ago      209MB
centos              latest           300e315adb2f      7 months ago      209MB
itying/centos       latest           300e315adb2f      7 months ago      209MB
itying/centos       v1.0.1           300e315adb2f      7 months ago      209MB
```

3.1.6、docker rmi 删除镜像

查看本地镜像

```
[root@localhost /]# docker images | grep centos
itying/centos       latest           300e315adb2f      7 months ago      209MB
itying/centos       v1.0.1           300e315adb2f      7 months ago      209MB
centos              8.3.2011         300e315adb2f      7 months ago      209MB
centos              latest           300e315adb2f      7 months ago      209MB
```

```
[root@localhost /]# docker rmi docker.io/itying/centos:latest
Untagged: itying/centos:latest
```

这样删除只会删除对应的标签

```
[root@localhost /]# docker images | grep centos
itying/centos       v1.0.1           300e315adb2f      7 months ago      209MB
centos              8.3.2011         300e315adb2f      7 months ago      209MB
centos              latest           300e315adb2f      7 months ago      209MB
```

要删除 imagesID 为 300e315adb2f 的镜像需要通过 id 删除

```
[root@localhost /]# docker rmi -f 300e315adb2f
Untagged: itying/centos:v1.0.1
Untagged: centos:8.3.2011
```

```
Untagged: centos:latest
Untagged: centos@sha256:5528e8b1b1719d34604c87e11dcd1c0a20bedf46e83b5632cdeac91b8c04efc1
Deleted: sha256:300e315adb2f96afe5f0b2780b87f28ae95231fe3bdd1e16b9ba606307728f55
Deleted: sha256:2653d992f4ef2bfd27f94db643815aa567240c37732cae1405ad1c1309ee9859
[root@localhost /]# docker images | grep centos
```

这样就会删除 imagesID 为 300e315adb2f 的 centos 镜像

3.1.7、把本地镜像推送到 dockerHub 仓库

一、需要在 dockerHub 上面注册一个账户：https://hub.docker.com/

二、使用 docker login 本地登录 dockerHub

```
docker login
```

```
[root@localhost ~]# docker login
Login with your Docker ID to push and pull images from Docker Hub. If you don't have a Docker ID, head over to https://hub.docker.com to create one.
Username: itying
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

查看保存的账户信息

```
[root@localhost ~]# cat .docker/config.json
{
    "auths": {
        "https://index.docker.io/v1/": {
            "auth": "aXR5aW5nOnpoYW5nxxG9uZzEyMzQ="
        }
```

```
    }
}
```

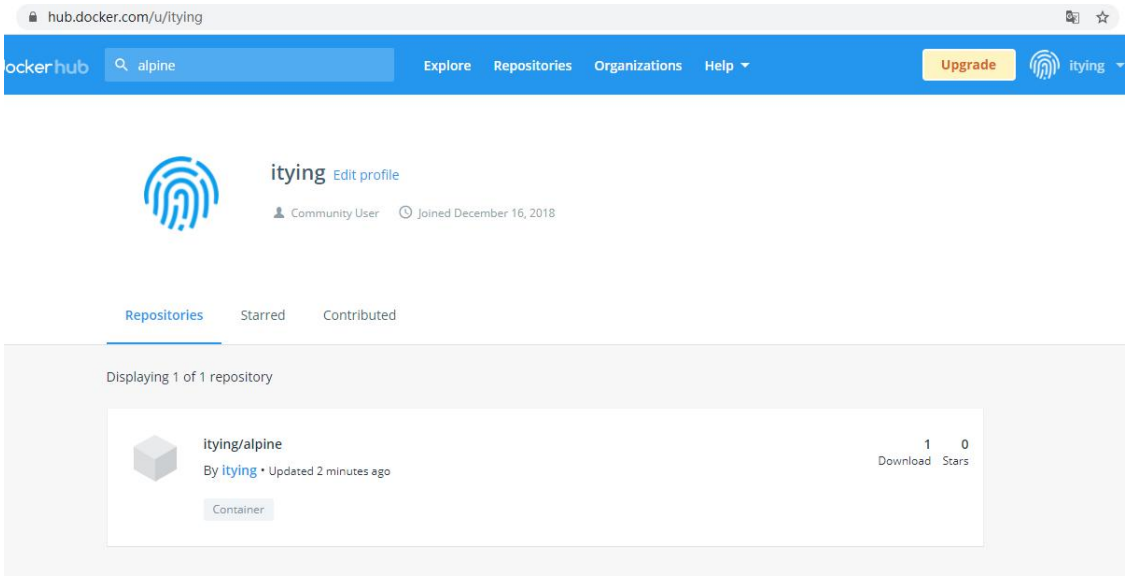
三、docker pull 镜像名称 把本地镜像推送到远程

下载一个 alpine 的镜像演示，alpine 是一个比较小的的 linux 镜像。

```
docker pull alpine

docker tag d4ff818577bc docker.io/itying/alpine:v1.0.1

docker tag d4ff818577bc docker.io/itying/alpine:latest

[root@localhost ~]# docker images | grep alpine
itying/alpine     v1.0.1      d4ff818577bc     4 weeks ago      5.6MB
alpine            3.14.0      d4ff818577bc     4 weeks ago      5.6MB
alpine            latest      d4ff818577bc     4 weeks ago      5.6MB
```

```
docker push docker.io/itying/alpine:v1.0.1
```

```
[root@localhost ~]# docker push docker.io/itying/alpine:v1.0.1
The push refers to repository [docker.io/itying/alpine]
72e830a4dff5: Mounted from library/alpine
v1.0.1: digest: sha256:1775bebec23e1f3ce486989bfc9ff3c4e951690df84aa9f926497d82f2ffca9
d size: 528
```



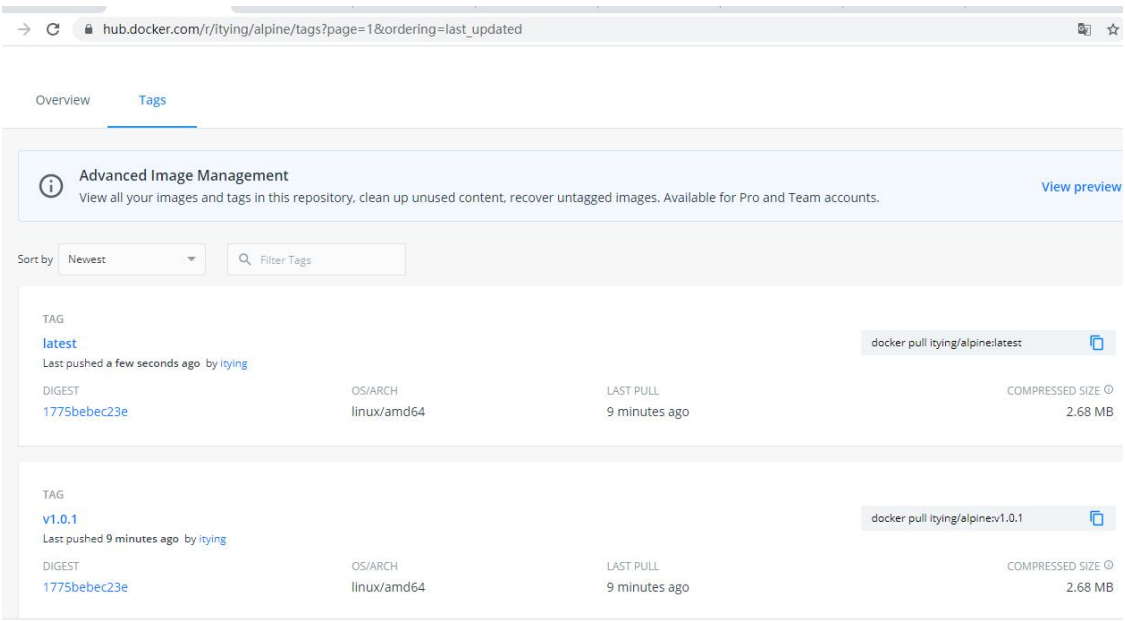
在打一个标签

```
[root@localhost ~]# docker images | grep alpine
itying/alpine      v1.0.1      d4ff818577bc    4 weeks ago      5.6MB
alpine             3.14.0      d4ff818577bc    4 weeks ago      5.6MB
alpine             latest      d4ff818577bc    4 weeks ago      5.6MB
```

```
docker tag d4ff818577bc docker.io/itying/alpine:latest
```

```
docker push docker.io/itying/alpine:latest
```

```
[root@localhost ~]# docker push docker.io/itying/alpine:latest
The push refers to repository [docker.io/itying/alpine]
72e830a4dff5: Layer already exists
Head https://production.cloudflare.docker.com/registry-v2/docker/registry/v2/blobs/sha256/d4/d4ff818577bc193b309b355b02ebc9220427090057b54a59e73b79bdfe139b83/data?verify=1626683919-RTFSjxvIO10s%2BetZ4bmxfWY%2BcvQ%3D: EOF
```



3.2、Docker 容器

类似 linux 系统环境，运行和隔离应用。容器从镜像启动的时候，docker 会在镜像的最上一层创建一个可写层，镜像本身是只读的，保持不变。容器与镜像的关系，就如同面向编程中对象与类之间的关系。

因为容器是通过镜像来创建的，所以必须先有镜像才能创建容器，而生成的容器是一个独立于宿主机的隔离进程，并且有属于容器自己的网络和命名空间。

3.2.1、查看所的容器

查看所有运行的容器

```
docker  ps
```

查看所有容器

```
docker  ps  -a
```

```
[root@localhost ~]# docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS
PORTS NAMES
36acd67c3ca7 hello-world "/hello" 3 hours ago Exited (0) 3 hours ago
hardcore_allen
docker run hello-world
[root@localhost ~]# docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS
PORTS NAMES
0ddf7480e541 hello-world "/hello" 3 seconds ago Exited (0) 3 seconds ago
blissful_austin
36acd67c3ca7 hello-world "/hello" 3 hours ago Exited (0) 3 hours ago
hardcore_allen
```

3.2.2、docker run 参数

docker run ： 创建一个新的容器并运行一个命令

docker run 是日常用的最频繁用的命令之一，同样也是较为复杂的命令之一
命令格式：docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

OPTIONS :选项

-i :表示启动一个可交互的容器，并持续打开标准输入

-t: 表示使用终端关联到容器的标准输入输出上

-d :表示容器放置后台运行

--rm:退出后即删除容器

--name :表示定义容器唯一名称

-p 映射端口

-v 指定路径挂载数据卷

-e 运行容器传递环境变量

IMAGE :表示要运行的镜像

COMMAND :表示启动容器时要运行的命令

3.2.3、-it 启动一个交互式容器

docker run 启动一个交互式容器在容器内执行/bin/bash 命令

```
[root@localhost ~]# docker run -it itying/centos /bin/bash
```

```
[root@4cff77aa531f /]# ls
bin  etc   lib        lost+found  mnt  proc  run   srv  tmp  var
dev  home  lib64      media       opt  root  sbin  sys  usr
[root@localhost ~]# docker tag 300e315adb2f docker.io/itying/centos
[root@localhost ~]# docker images
REPOSITORY          TAG           IMAGE ID          CREATED           SIZE
mysql               latest        c60d96bd2b77      4 days ago        514MB
itying/alpine       latest        d4ff818577bc      5 weeks ago       5.6MB
itying/alpine       v1.0.1        d4ff818577bc      5 weeks ago       5.6MB
alpine              latest        d4ff818577bc      5 weeks ago       5.6MB
hello-world         latest        d1165f221234      4 months ago      13.3kB
itying/centos       latest        300e315adb2f      7 months ago      209MB
centos              latest        300e315adb2f      7 months ago      209MB
```

```
[root@localhost ~]# docker run -it itying/centos /bin/bash
[root@4cff77aa531f /]# ls
bin  etc   lib        lost+found  mnt  proc  run   srv  tmp  var
dev  home  lib64      media       opt  root  sbin  sys  usr
```

```
[root@4cff77aa531f /]# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defa
ult qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
4: eth0@if5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP gr
oup default
    link/ether 02:42:ac:11:00:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
       valid_lft forever preferred_lft forever
```

退出容器

exit：容器停止并退出

ctrl+p+q：容器不停止退出

```
[root@4cff77aa531f /]# exit
exit
```

```
[root@localhost ~]# docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS
PORTS NAMES
4cff77aa531f itying/centos "/bin/bash" 3 minutes ago Exited (0) 27 seconds ago
xenodochial_euler
9170ef3ad0fd hello-world "/hello" 6 days ago Exited (0) 6 days ago
tender_einstein
b05c85ce55d1 itying/alpine "/bin/sh" 6 days ago Exited (0) 6 days ago
gallant_mclean
```

3.2.4、--rm 启动一个退出即删除容器

创建一个非交互式的容器用完就删除

```
docker run --rm itying/centos /bin/echo hello itying

[root@localhost ~]# docker run --rm itying/centos /bin/echo hello itying
hello itying
```

创建一个交互式的容器用完就删

```
[root@localhost ~]# docker run -it --rm itying/centos /bin/bash

[root@6aac2c6f545f /]# ls
bin etc lib lost+found mnt proc run srv tmp var
dev home lib64 media opt root sbin sys usr
[root@6aac2c6f545f /]# exit
exit

[root@localhost ~]# docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS
```

```
                    PORTS          NAMES
4cff77aa531f    itying/centos    "/bin/bash"    4 minutes ago    Exited (0) About a minute
 ago                    xenodochial_euler
9170ef3ad0fd    hello-world      "/hello"       6 days ago       Exited (0) 6 days ago
                    tender_einstein
b05c85ce55d1    itying/alpine    "/bin/sh"      6 days ago       Exited (0) 6 days ago
                    gallant_mclean
```

3.2.5、-d 启动一个后台容器

开启一个后台交互的容器，注意启动的时候加上-it,如果不加的话 docker 启动容器会自动停止。

```
docker run -ti -d itying/centos
```

```
[root@localhost ~]# docker ps
CONTAINER ID    IMAGE              COMMAND          CREATED                STATUS
              PORTS          NAMES
93200113fb7e    itying/centos    "/bin/bash"    About a minute ago    Up About a minute
                clever_beaver
```

3.2.6、exec 进入置为后台已经启动的容器

进入一个容器可以使用 docker exec 或者 docker attach

docker exec

交互式方式进入容器，容器内执行/bin/bash 命令

```
[root@localhost ~]# docker exec --help

Usage:  docker exec [OPTIONS] CONTAINER COMMAND [ARG...]

Run a command in a running container

Options:
  -d, --detach                Detached mode: run command in the background
      --detach-keys string    Override the key sequence for detaching a container
```

```
  -e, --env list                Set environment variables
      --env-file list           Read in a file of environment variables
  -i, --interactive             Keep STDIN open even if not attached
      --privileged              Give extended privileges to the command
  -t, --tty                     Allocate a pseudo-TTY
  -u, --user string             Username or UID (format: <name|uid>[:<group|gid>])
  -w, --workdir string          Working directory inside the container
docker exec -ti 93200113fb7e /bin/sh
docker exec -ti 93200113fb7e /bin/bash
```

docker attach

```
[root@localhost ~]# docker attach --help

Usage:  docker attach [OPTIONS] CONTAINER

Attach local standard input, output, and error streams to a running container

Options:
      --detach-keys string   Override the key sequence for detaching a container
      --no-stdin             Do not attach STDIN
      --sig-proxy            Proxy all received signals to the process (default true)
```

```
[root@localhost ~]# docker attach 6f0f53beab62
```

区别：

docker exec：进入容器开启一个新的终端（常用） 执行 exit 退出的时候不会停止容器

docker attach：进入容器正在执行的终端 exit 退出会停止容器

测试

/bin/sh -c -c 命令表示后面的参数将会作为字符串读入作为执行的命令

```
[root@localhost ~]# docker run -d centos /bin/sh -c "while true;do echo itying;sleep 1;do
ne"
7081e7433f5c0565a99ee0bd0d0eb5429b17b7b06c966ba544924601f4a78c6c
[root@localhost ~]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS
          PORTS     NAMES
7081e7433f5c   centos    "/bin/sh -c 'while t…"   10 seconds ago   Up 9 seconds
            exciting_leavitt
[root@localhost ~]# docker logs -tf 7081e7433f5c
2021-07-27T03:45:08.531550546Z itying
2021-07-27T03:45:09.883724427Z itying
2021-07-27T03:45:10.887146984Z itying
2021-07-27T03:45:11.888594483Z itying
[root@localhost ~]# docker exec -it 7081e7433f5c /bin/bash
[root@7081e7433f5c /]#
[root@localhost ~]# docker attach 7081e7433f5c
itying
itying
itying
```

3.2.7 --name 启动容器的时候指定名称

docker run -it --name MyCentos itying/centos /bin/bash

```
[root@localhost ~]# docker ps
CONTAINER ID   IMAGE           COMMAND       CREATED          STATUS          PORT
S     NAMES
095f45520b1e   itying/centos   "/bin/bash"   About a minute ago              MyC
entos
```

3.2.8、start 启动 stop 停止 restart 重启容器 exit 退出容器

```
docker start 07fb9c32ea61
docker stop 07fb9c32ea61
docker restart 07fb9c32ea61
docker restart MyCentos
```

退出容器

exit：容器停止并退出

ctrl+p+q：容器不停止退出

3.2.9、删除容器

```
docker rm myLinux
docker rm -f myLinux
```

删除所有容器

```
docker rm -f $(docker ps -q)
```

根据容器的状态，删除 Exited 状态的容器

```
docker rm $(docker ps -qf status=exited)
```

3.2.10、查看容器操作日志

docker logs：获取容器的日志

语法
docker logs [OPTIONS] CONTAINER

OPTIONS 说明：

-f：跟踪日志输出

--since :显示某个开始时间的所有日志

-t：显示时间戳

--tail :仅列出最新 N 条容器日

```
docker logs 9170ef3ad0fd
docker logs -f mynginx
[root@localhost ~]# docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS
PORTS NAMES
```

```
9170ef3ad0fd    hello-world    "/hello"    3 minutes ago    Exited (0) 3 minutes ago
                tender_einstein
b05c85ce55d1    itying/alpine  "/bin/sh"   38 minutes ago   Exited (0) 37 minutes ago
                gallant_mclean

[root@localhost ~]# docker logs -f 9170ef3ad0fd

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

```
[root@localhost ~]# docker ps -a
CONTAINER ID    IMAGE           COMMAND     CREATED          STATUS
                PORTS     NAMES
9170ef3ad0fd    hello-world     "/hello"    3 minutes ago    Exited (0) 3 minutes ago
                tender_einstein
b05c85ce55d1    itying/alpine   "/bin/sh"   38 minutes ago   Exited (0) 37 minutes ago
                gallant_mclean
```

```
[root@localhost ~]# docker logs b05c85ce55d1
```

```
/ # ls
bin   etc   lib   mnt   proc   run   srv   tmp   var
dev   home   media   opt   root   sbin   sys   usr
/ # cd root
~ # ls
~ # echo hello>1.txt
~ # ls
1.txt
~ # cat 1.txt
hello
~ # exit
```

3.2.11、docker commit 容器转换为镜像

镜像是没有写入权限的，但是我们可以修改容器把容器制作为镜像

启动一个容器 给容器写入内容

```
[root@localhost ~]# docker run -ti --name myLinux itying/alpine /bin/sh
~ # ls
bin   etc   lib   mnt   proc   run   srv   tmp   var
dev   home   media   opt   root   sbin   sys   usr
~ # cd root/
~ # echo hello itying > 1.txt
~ # ls
1.txt
~ # cat 1.txt
hello itying
~ # exit
```

```
[root@localhost ~]# docker ps -a
CONTAINER ID   IMAGE              COMMAND       CREATED             STATUS
              PORTS       NAMES
d4486a7dff1b   itying/alpine   "/bin/sh"   2 minutes ago   Exited (0) 2 minutes ago
              myLinux
```

```
docker commit d4486a7dff1b mylinux_1.txt
```

```
[root@localhost ~]# docker images
REPOSITORY          TAG             IMAGE ID            CREATED             SIZE
mylinux_1.txt       latest          75df7ca78e85        4 seconds ago       5.6MB
alpine              latest          d4ff818577bc        4 weeks ago         5.6MB
itying/alpine       latest          d4ff818577bc        4 weeks ago         5.6MB


[root@localhost ~]# docker run -ti 75df7ca78e85 /bin/sh
/ # cd root/
~ # cat 1.txt
hello itying
```

3.2.12、镜像的导入导出

我们可以把制作好的镜像发布到 dockerHub，我们也可以把制作好的镜像导出发给别人。

一、镜像导出

```
[root@localhost ~]# docker images
REPOSITORY                  TAG             IMAGE ID            CREATED             SIZE
itying/alpine_1.txt         latest          9f41cdfdaafd        10 minutes ago      5.6MB
alpine                      latest          d4ff818577bc        4 weeks ago         5.6MB
itying/alpine               latest          d4ff818577bc        4 weeks ago         5.6MB
```

```
docker save 9f41cdfdaafd  > itying.alpine_1.txt.tar
[root@localhost ~]# ls
itying.alpine_1.txt.tar
```

二、镜像导入

```
[root@localhost ~]# docker images
REPOSITORY                  TAG             IMAGE ID            CREATED             SIZE
```

```
itying/alpine_1.txt     latest      9f41cdfdaafd     12 minutes ago     5.6MB
itying/alpine           latest      d4ff818577bc     4 weeks ago        5.6MB
alpine                  latest      d4ff818577bc     4 weeks ago        5.6MB
```

```
[root@localhost ~]# docker rmi -f itying/alpine_1.txt
Untagged: itying/alpine_1.txt:latest
Deleted: sha256:9f41cdfdaafdf2af3a587ba00ccc837772d0a800eae3429672fd84637c027612
Deleted: sha256:2958bb14f2bd3a1c677e50a6008d2bfd51a543b2d7357633485618150e5149e8
```

```
[root@localhost ~]# docker images
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
itying/alpine     latest      d4ff818577bc     4 weeks ago     5.6MB
alpine              latest      d4ff818577bc     4 weeks ago     5.6MB
```

```
[root@localhost ~]# docker load < itying.alpine_1.txt.tar
4495f766bc01: Loading layer  3.584kB/3.584kB
Loaded image ID: sha256:9f41cdfdaafdf2af3a587ba00ccc837772d0a800eae3429672fd84637c0
27612
```

```
[root@localhost ~]# docker images
REPOSITORY          TAG          IMAGE ID          CREATED             SIZE
<none>              <none>      9f41cdfdaafd     16 minutes ago     5.6MB
itying/alpine     latest      d4ff818577bc     4 weeks ago        5.6MB
alpine              latest      d4ff818577bc     4 weeks ago        5.6MB
```

```
[root@localhost ~]# docker tag 9f41cdfdaafd itying/alpine.1.txt:latest
[root@localhost ~]# docker images
REPOSITORY                TAG          IMAGE ID          CREATED             SIZE
itying/alpine.1.txt     latest      9f41cdfdaafd     17 minutes ago     5.6MB
itying/alpine           latest      d4ff818577bc     4 weeks ago        5.6MB
alpine                  latest      d4ff818577bc     4 weeks ago        5.6MB
```

验证

```
[root@localhost ~]# docker run --rm -ti itying/alpine_1.txt /bin/sh
~ # ls
bin      etc      lib      mnt      proc     run      srv      tmp      var
dev      home     media    opt      root     sbin     sys      usr
~ # cd root/
~ # ls
1.txt
```

3.2.13、docker cp 实现数据拷贝

docker cp :用于容器与主机之间的数据拷贝。

语法：

```
docker cp [OPTIONS] CONTAINER:SRC_PATH DEST_PATH

docker cp [OPTIONS] SRC_PATH  CONTAINER:DEST_PATH
```

示例：

将主机/www/itying 目录拷贝到容器 5e324512adf1 的/root 目录下。

```
docker cp /root/ityingwwwroot/ 5e324512adf1:root
```

将主机/root/ityingwwwroot 目录拷贝到容器 5e324512adf1 中，目录重命名为 wwwroot。

```
docker cp /root/ityingwwwroot/  5e324512adf1:root/wwwroot
```

将容器 5e324512adf1 的/root/wwwroot 目录拷贝到主机的/root 目录中。

```
[root@localhost ~]# docker cp 5e324512adf1:root/wwwroot  /root

[root@localhost ~]# ls
```

```
anaconda-ks.cfg    wwwroot
```

将容器一个目录中的所有文件复制到容器的 root/aaa 目录

```
docker cp . 5e324512adf1:root/aaa
```

完整示例：

```
[root@localhost ~]# docker ps
CONTAINER ID    IMAGE              COMMAND                  CREATED          STATU
S          PORTS                                    NAMES
e3488b28d60e    08b152afcfae       "/docker-entrypoint.…"   3 hours ago      Up 3 hours
0.0.0.0:81->80/tcp, :::81->80/tcp   jolly_shirley
095f45520b1e    itying/centos      "/bin/bash"              4 hours ago      Up 4 hours
                                   MyCentos
```

```
[root@localhost ~]# docker cp /root/myzip.zip 095f45520b1e:/root/
[root@localhost ~]# docker exec -it 095f45520b1e /bin/bash
[root@095f45520b1e /]# cd root/
[root@095f45520b1e ~]# ls
anaconda-ks.cfg   anaconda-post.log   myzip.zip   original-ks.cfg
```

```
[root@localhost mnt]# docker cp 095f45520b1e:/root/myzip.zip /mnt
[root@localhost mnt]# ls
myzip.zip
```

3.4、Docker 部署 Nginx 以及端口映射

Docker 部署 Nginx

https://hub.docker.com/_/nginx

3.4.1、下载 nginx 镜像

```
docker pull nginx
```

```
[root@localhost ~]# docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
33847f680f63: Already exists
dbb907d5159d: Pull complete
8a268f30c42a: Pull complete
b10cf527a02d: Pull complete
c90b090c213b: Pull complete
1f41b2f2bf94: Pull complete
Digest: sha256:8f335768880da6baf72b70c701002b45f4932acae8d574dedfddaf967fc3ac90
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
[root@localhost ~]# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
nginx               latest              08b152afcfae        4 days ago          133MB
```

打个标签

```
[root@localhost ~]# docker tag 08b152afcfae docker.io/itying/nginx:latest
[root@localhost ~]# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
itying/nginx        latest              08b152afcfae        4 days ago          133MB
```

3.4.2、 运行 nginx 容器

```
[root@localhost ~]# docker run -it -d --rm --name Mynginx itying/nginx
```

```
73fc9513cbbf4cbf3e5749241438afc4580931f1e92add418769de4d3f847da3
```

查看容器

```
[root@localhost ~]# docker ps
CONTAINER ID    IMAGE              COMMAND                  CREATED             ST
ATUS              PORTS       NAMES
73fc9513cbbf    itying/nginx     "/docker-entrypoint.…"    4 seconds ago      Up 2 seconds
    80/tcp      Mynginx
```

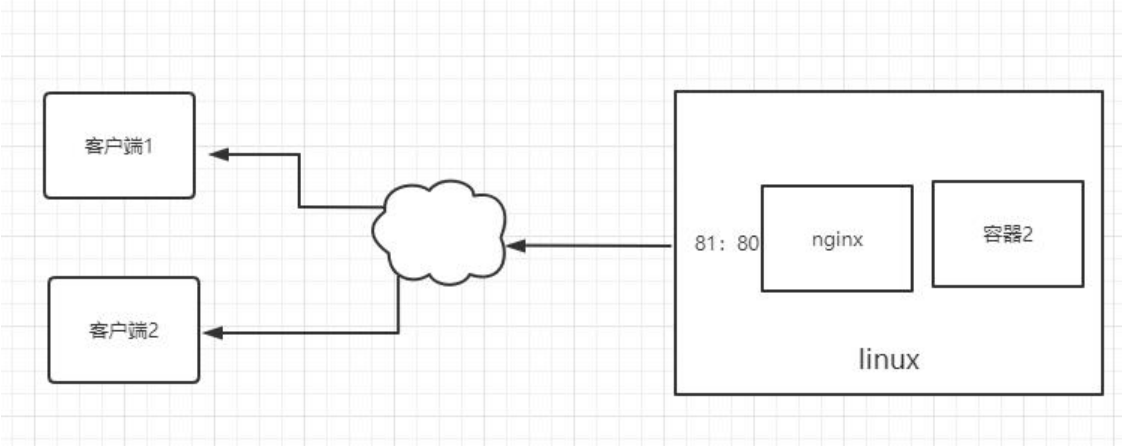
3.4.3 、进入容器访问 url

```
[root@localhost ~]# docker ps
CONTAINER ID    IMAGE              COMMAND                  CREATED             ST
ATUS              PORTS       NAMES
73fc9513cbbf    itying/nginx     "/docker-entrypoint.…"    4 seconds ago      Up 2 seconds
    80/tcp      Mynginx
095f45520b1e    itying/centos    "/bin/bash"               43 minutes ago     Up 43 minu
tes                 MyCentos

[root@localhost ~]# docker exec -ti 73fc9513cbbf /bin/bash

root@73fc9513cbbf:/# curl 127.0.0.1
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
```

```
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

3.4.4、-p 映射端口

我们想在外部访问容器里面的 nginx 这个时候我们就需要映射端口



我们可以通过 -p 参数来映射端口

基本语法：

docker run -p 容器外端口:容器内端口 容器 ID

```
docker run -ti -d --rm -p 81:80 08b152afcfae
```

```
[root@localhost ~]# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
itying/nginx        latest              08b152afcfae        4 days ago          133MB
```

```
[root@localhost ~]# docker run -ti -d --rm -p 81:80 08b152afcfae
CONTAINER ID   IMAGE          COMMAND                  CREATED
  STATUS                                  PORTS                                   NAMES
e3488b28d60e   08b152afcfae   "/docker-entrypoint.…"   About a minute ago   Up Ab
out a minute                  0.0.0.0:81->80/tcp, :::81->80/tcp   jolly_shirley

[root@localhost ~]# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defa
ult qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP grou
p default qlen 1000
    link/ether 00:0c:29:3e:06:99 brd ff:ff:ff:ff:ff:ff
    inet 192.168.241.128/24 brd 192.168.241.255 scope global dynamic noprefixroute ens
33
       valid_lft 1725sec preferred_lft 1725sec
    inet6 fe80::20c:29ff:fe3e:699/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

外部电脑上面通过 81 访问

http://192.168.241.128:81/

3.5、Docker 部署 Nginx 挂载数据卷

3.5.1、-v 指定路径挂载数据卷

基本语法：

```
docker run -v  容器外目录:容器内目录  容器 ID
```

首先创建一个目录

```
[root@localhost ~]# mkdir wwwroot
[root@localhost ~]# ls
```

```
aaa.txt  anaconda-ks.cfg  httpd.conf  index.html  wwwroot
```

其次把百度的 index.html 下载到本地

```
[root@localhost wwwroot]# wget www.baidu.com -O index.html
--2021-07-20 03:38:43--  http://www.baidu.com/
正在解析主机 www.baidu.com (www.baidu.com)... 182.61.200.7, 182.61.200.6
正在连接 www.baidu.com (www.baidu.com)|182.61.200.7|:80... 已连接。
已发出 HTTP 请求，正在等待回应... 200 OK
长度：2381 (2.3K) [text/html]
正在保存至: “index.html”

index.html                                          100%[=======================================
============================>]    2.33K  --.-KB/s  用时 0s

2021-07-20 03:38:43 (5.08 MB/s) - 已保存 “index.html” [2381/2381])

[root@localhost wwwroot]# ls
index.html


[root@localhost wwwroot]# cat index.html
```

```
docker run -ti -d --rm --name nginx_with_baidu -p 82:80 -v /root/wwwroot:/usr/share/ngi
nx/html itying/nginx:v1.0.0

docker exec -ti 3408e7d7a430 /bin/bash
```

3.5.2、-v 匿名挂载数据卷

基本语法：

```
docker run -v 容器内目录  容器 ID
```

-P 表示随机映射端口

```
docker run -d -P --name nginx01 -v /etc/nginx nginx
```

```
[root@localhost ~]# docker run -d -P --name nginx01 -v /etc/nginx nginx
e6aea0a9c58cf4f31ce0ebdd43b071845ba5b1ee772949a5cf02da5f40e2245c
[root@localhost ~]# docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATU
S PORTS NAMES
e6aea0a9c58c nginx "/docker-entrypoint…." 2 seconds ago Up 1 second
0.0.0.0:49155->80/tcp, :::49155->80/tcp nginx01
```

http://192.168.241.128:49155/

查看挂载详情：

```
[root@localhost ~]# docker volume ls
DRIVER VOLUME NAME
local 3f55b08bbafb39845c98efe1cf18dfa7fedea21c2fad24cb14759dced3872ba2
local 4de46af05120a375cb5270af59790f432c64758517352963f55e67cd3bfa56d8
local 5a8a57ea0fe775429577c451d23687f76634b5ee845c95e6e30a099b888ac2c1
local 5c97447224de838eab94272de4cb556cea8e79a4469b618c00cba1b826e0d37e
local 6b8cfbcf9727dc2c3f404cf9d830de7971fe7d46a56c5a0a42222182917fd6ae
local 6b9adae890deb91efcf5a52913f6ab61d8c22a83c2130fc21c9a3f44f32919
```

```
[root@localhost ~]# docker inspect e6aea0a9c58c | grep volume
"Type": "volume",
"Source": "/var/lib/docker/volumes/3f55b08bbafb39845c98efe1cf18dfa7fed
ea21c2fad24cb14759dced3872ba2/_data"
```

3.5.3、-v 具名挂载数据卷

基本语法

```
docker run -v 卷名称:容器内目录  容器 ID
```

```
docker run -d -P --name nginx01 -v juming:/etc/nginx nginx
```

```
[root@localhost ~]# docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED              STATUS              PORTS                                                  NAMES
57764c6d6901   nginx     "/docker-entrypoint.…"   5 seconds ago        Up 4 seconds        0.0.0.0:49156->80/tcp, :::49156->80/tcp                nginx01
d91b73c03473   mongo     "docker-entrypoint.s…"   About an hour ago    Up About an hour    0.0.0.0:27017->27017/tcp, :::27017->27017/tcp          authMongo
83c2eda7329d   redis     "docker-entrypoint.s…"   5 hours ago          Up 5 hours          0.0.0.0:6379->6379/tcp, :::6379->6379/tcp              redis-test
f6dbc4920782   mysql     "docker-entrypoint.s…"   5 hours ago          Up 5 hours          0.0.0.0:3306->3306/tcp, :::3306->3306/tcp, 33060/tcp   ityingMysql
```

```
[root@localhost ~]# docker volume ls
DRIVER    VOLUME NAME
local     3f55b08bbafb39845c98efe1cf18dfa7fedea21c2fad24cb14759dced3872ba2
local     fdf7b0c7847f62ffe2326ef8a11b746efbb8b9fd212970cd31ccca5385fc9865
local     juming
```

查看挂载详情

```
[root@localhost ~]# docker volume inspect juming
[
    {
        "CreatedAt": "2021-07-28T05:09:13-04:00",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/juming/_data",
```

```
        "Name": "juming",
        "Options": null,
        "Scope": "local"
    }
]
```

3.5.5、docker inspect 命令查看容器运行的细节

查看详细的映射关系

```
[root@localhost wwwroot]# docker inspect 3408e7d7a430 | grep share

                "/root/wwwroot:/usr/share/nginx/html"
                "Destination": "/usr/share/nginx/html",
```

3.6、Docker 运行容器传递环境变量

语法：

docker run -e 变量名=变量值 容器 ID

```
[root@localhost /]# docker run --rm -e E_OPTS=abacefg itying/alpine printenv
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=8e0aa8df2905
E_OPTS=abacefg
HOME=/root
```

```
[root@localhost /]# docker run --rm itying/alpine printenv
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=480941540dab
HOME=/root
```

传递多个环境变量

```
[root@localhost /]# docker run --rm -e E_OPTS=abacefg -e Android=aaaaaaaaaaaa itying/alpine printenv
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

```
HOSTNAME=f86ce3f51d67
E_OPTS=abacefg
Android=aaaaaaaaaaaa
HOME=/root
```

3.7、Docker 容器内安装软件

Centos 系列的 linux 安装软件

Docker 容器内安装软件和以前 linux 中安装软是一样的。

```
[root@localhost ~]# docker exec -ti MyCentos /bin/bash
[root@095f45520b1e /]# ls
bin dev etc home lib lib64 lost+found media mnt opt proc root run sbin
 srv sys tmp usr var
[root@095f45520b1e /]# ifconfig
bash: ifconfig: command not found
```

安装 net-tools

```
 yum install -y net-tools
```

查看容器的 ip

```
[root@095f45520b1e /]# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.17.0.2  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 02:42:ac:11:00:02  txqueuelen 0  (Ethernet)
        RX packets 7805  bytes 14138450 (13.4 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 4715  bytes 259714 (253.6 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
```

```
        TX  errors  0    dropped  0  overruns  0    carrier  0    collisions  0
```

Ubuntu 系列的 linux 安装软件

```
apt-get  update
apt-get  install  -y  vim
```

四、Docker 应用实例

4.1、安装 nodejs

https://hub.docker.com/_/node

下载镜像：

```
[root@localhost /]# docker pull node
Using default tag: latest
latest: Pulling from library/node
627b765e08d1: Downloading      23.6MB/50.44MB
627b765e08d1: Pull complete

ec88359bd66f: Pull complete
1829abc8bab1: Pull complete
47f423bb50ee: Pull complete
Digest: sha256:8229a1f3580d32fa18b2304fa23df6e9e3d53fdb958fd8ffe812ca7a0a26bb69
Status: Downloaded newer image for node:latest
docker.io/library/node:latest
```

```
[root@localhost /]# docker images
REPOSITORY          TAG             IMAGE ID            CREATED             SIZE
node                latest          8f1b7f0dfc2f        4 days ago          907MB
itying/nginx        latest          08b152afcfae        4 days ago          133MB
nginx               latest          08b152afcfae        4 days ago          133MB
```

创建容器：

```
docker run -itd --name myNode -p 3000:3000 -v /root/wwwroot:/root/wwwroot node:latest
```

```
[root@localhost ~]# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
451560dfd59f itying/nginx "/docker-entrypoint.…." 17 minutes ago Up 17 minutes 0.0.0.0:81->80/tcp, :::81->80/tcp myNginx
6827f80b95e1 node "docker-entrypoint.s…" 55 minutes ago Up 9 minutes 0.0.0.0:3000->3000/tcp, :::3000->3000/tcp myNodejs
```

```
[root@localhost /]# docker exec -it myNode /bin/bash
root@b27322a23a2c:/# node -v
v16.5.0
```

挂载目录运行 nodejs 程序以及目录

```
[root@localhost /]# docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
node latest 8f1b7f0dfc2f 4 days ago 907MB
itying/nginx latest 08b152afcfae 4 days ago 133MB
```

启动容器:

```
docker run -itd --name myNodejs -p 3000:3000 -v /root/wwwroot:/root/ node:latest
```

查看镜像：

```
[root@localhost /]# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS
PORTS NAMES
6827f80b95e1 node "docker-entrypoint.s…" 50 seconds ago Up 49 seconds
0.0.0.0:3000->3000/tcp, :::3000->3000/tcp myNodejs
[root@localhost /]# docker inspect 6827f80b95e1 | grep wwwroot
"/root/wwwroot:/root/"
"Source": "/root/wwwroot"
```

进入容器：

```
docker exec -ti 6827f80b95e1 /bin/bash
```

安装 cnpm:

```
npm install cnpm -g --registry=https://registry.nlark.com
```

后台运行程序:

```
nohup node app.js &
```

访问：

http://192.168.241.128:3000/

查看进程：

```
root@6827f80b95e1:/# ps -aux
USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
root 1 0.0 2.6 417580 35608 pts/0 Ssl+ 10:12 0:00 node
root 18 0.0 0.2 4000 3316 pts/1 Ss+ 10:14 0:00 /bin/bash
root 40 0.0 0.2 4004 3248 pts/2 Ss 10:16 0:00 /bin/bash
root 58 1.2 3.2 586792 43360 pts/1 Sl 10:17 0:00 node app.js
root 65 0.0 0.2 7644 2692 pts/2 R+ 10:17 0:00 ps -aux
```

杀死进程：

```
root@6827f80b95e1:/# kill -9 58

root@6827f80b95e1:/# ps -aux
USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
root 1 0.0 2.6 417580 35608 pts/0 Ssl+ 10:12 0:00 node
root 18 0.0 0.2 4000 3316 pts/1 Ss+ 10:14 0:00 /bin/bash
root 40 0.0 0.2 4004 3248 pts/2 Ss 10:16 0:00 /bin/bash
root 66 0.0 0.2 7644 2668 pts/2 R+ 10:18 0:00 ps -aux
```

4.2、安装 mysql

https://hub.docker.com/_/mysql

下载 mysql

```
docker pull mysql
```

4.2.1 映射端口启动 mysql 远程连接

启动 mysql

```
docker run --name ityingmysql -p 3306:3306 -e MYSQL_ROOT_PASSWORD=123456 -d mysql
```

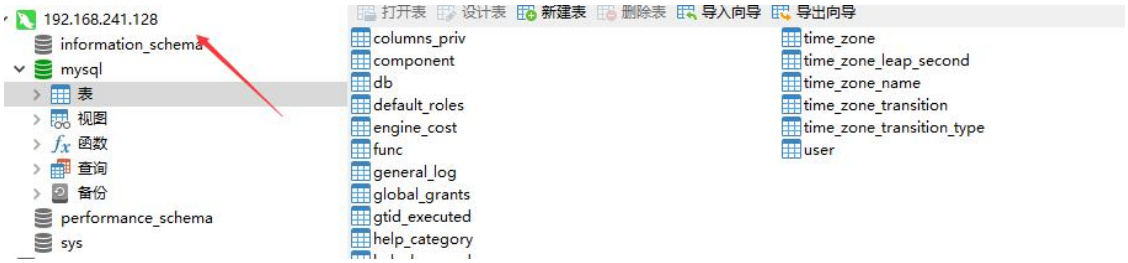
```
[root@localhost ~]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS
          PORTS                                                 NAMES
d280d5d97b7c   mysql     "docker-entrypoint.s…"   24 seconds ago   Up 22 seconds
 0.0.0.0:3306->3306/tcp, :::3306->3306/tcp, 33060/tcp   ityingmysql
```

进入 mysql 连接 mysql

```
[root@localhost ~]# docker exec -it ityingmysql /bin/bash
root@d280d5d97b7c:/# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.26 MySQL Community Server - GPL

Copyright (c) 2000, 2021, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

```
mysql>
```

外部连接 Mysql



4.2.2 映射端口 、 挂载配置文件目录、挂载数据文件目录 启动 mysql

默认数据库的数据是放在容器里面的，这样的话当容器删除会导致数据丢失。我们想的是删除容器的时候不删除容器里面的 mysql 数据，这个时候启动容器的时候就可以把 mysql 数据挂载到外部。

```
docker run --name ityingMysql -p 3306:3306 -v /root/mysql/conf.d:/etc/mysql/conf.d -v /root/mysql/data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=123456 -d mysql
```

```
[root@localhost ~]# docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS
          PORTS                                                  NAMES
4d912093b257   mysql     "docker-entrypoint.s…"   29 seconds ago   Up 28 seconds
 0.0.0.0:3306->3306/tcp, :::3306->3306/tcp, 33060/tcp   ityingMysql
[root@localhost ~]# docker inspect 4d912093b257 | grep mysql
                "mysqld"
                    "/root/mysql/conf.d:/etc/mysql/conf.d",
                    "/root/mysql/data:/var/lib/mysql"
                    "Source": "/root/mysql/conf.d",
                    "Destination": "/etc/mysql/conf.d",
                    "Source": "/root/mysql/data",
                    "Destination": "/var/lib/mysql",
                    "mysqld"
                "Image": "mysql",
                    "/var/lib/mysql": {}
```

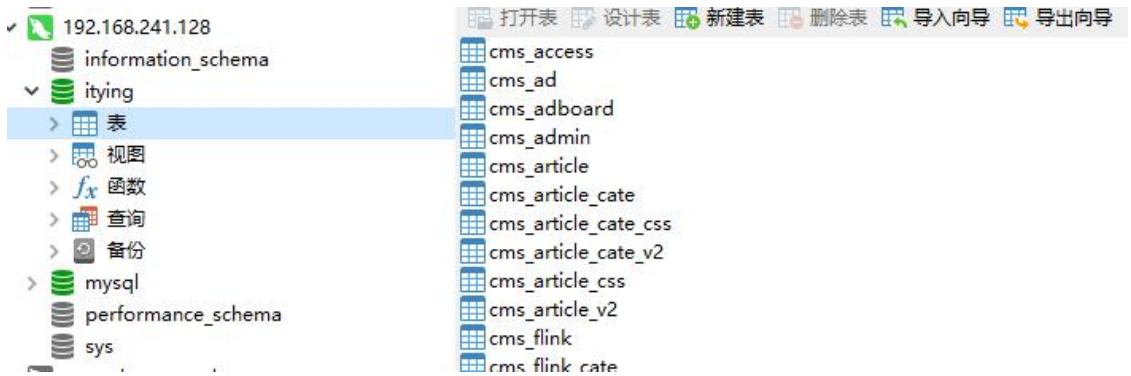
删除容器：

```
[root@localhost ~]# docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS
        PORTS                                                  NAMES
4d912093b257   mysql     "docker-entrypoint.s…"   2 minutes ago    Up 2 minutes    0.
0.0.0:3306->3306/tcp, :::3306->3306/tcp, 33060/tcp     ityingMysql
```

```
[root@localhost ~]# docker rm -f 4d912093b257
```

重新创建容器：

```
docker run --name ityingMysql -p 3306:3306 -v /root/mysql/conf.d:/etc/mysql/conf.d -v /ro
ot/mysql/data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=123456 -d mysql
```



4.3、docker 安装 Redis

https://hub.docker.com/_/redis

```
[root@localhost ~]# docker pull redis
Using default tag: latest
latest: Pulling from library/redis
33847f680f63: Already exists
26a746039521: Pull complete
18d87da94363: Pull complete
ecf0dbe7c357: Pull complete
46f280ba52da: Pull complete
Digest: sha256:cd0c68c5479f2db4b9e2c5fbfdb7a8acb77625322dd5b474578515422d3ddb59
Status: Downloaded newer image for redis:latest
```

```
docker run -itd --name redis-test -p 6379:6379 redis

[root@localhost ~]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS
          PORTS                                                  NAMES
83c2eda7329d   redis     "docker-entrypoint.s…"   9 seconds ago    Up 9 seconds
0.0.0.0:6379->6379/tcp, :::6379->6379/tcp                       redis-test
f6dbc4920782   mysql     "docker-entrypoint.s…"   17 minutes ago   Up 16 minutes
0.0.0.0:3306->3306/tcp, :::3306->3306/tcp, 33060/tcp   ityingMysql
```

4.3.1 、本地连接

docker exec -it redis-test /bin/bash

```
[root@localhost ~]# docker exec -it redis-test /bin/bash
root@83c2eda7329d:/data# redis-cli
127.0.0.1:6379> set username zhangsan
OK
127.0.0.1:6379> get username
"zhangsan"
```

4.3.2、 远程连接

如果需要在远程 redis 服务上执行命令，同样我们使用的也是 **redis-cli** 命令。

语法

```
$ redis-cli -h host -p port
```

-h 服务器地址 -p 端口号

```
C:\Users\itying>redis-cli -h 192.168.241.128 -p 6379
192.168.241.128:6379> get username
"zhangsan"
192.168.241.128:6379>
```

4.3.2、 启动 docker 容器配置密码

通过 --requirepass 可以配置密码

运行容器

```
docker run -tid --name redis02 -p 6378:6379 redis --requirepass "123456"
```

本地连接

```
root@82f0b496ec3d:/data# redis-cli
127.0.0.1:6379> auth 123456
OK
```

远程连接

注意映射的端口

```
C:\Users\htzhanglong>redis-cli -h 192.168.241.130 -p 6378
192.168.241.130:6378> auth 123456
OK
```

4.4、docker 安装 Mongodb

https://hub.docker.com/_/mongo

4.1.1、远程连接不需要密码

```
[root@localhost ~]# docker pull mongo
Using default tag: latest
latest: Pulling from library/mongo
16ec32c2132b: Pull complete
6335cf672677: Pull complete
4b9c6ac629be: Pull complete
4de7437f497e: Pull complete
Digest: sha256:d78c7ace6822297a7e1c7076eb9a7560a81a6ef856ab8d9cde5d18438ca9e8bf
Status: Downloaded newer image for mongo:latest
docker.io/library/mongo:latest
```

启动容器 映射端口 挂载目录

```
docker run --name ityingMongo -p 27017:27017 -v /root/mongo/data:/data/db -d mongo
```

```
[root@localhost ~]# docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS
          PORTS                                                  NAMES
f9b5dd026a4e   mongo     "docker-entrypoint.s…"   12 seconds ago   Up 11 seconds
 0.0.0.0:27017->27017/tcp, :::27017->27017/tcp                  ityingMongo
83c2eda7329d   redis     "docker-entrypoint.s…"   25 minutes ago   Up 25 minutes
0.0.0.0:6379->6379/tcp, :::6379->6379/tcp                      redis-test
f6dbc4920782   mysql     "docker-entrypoint.s…"   42 minutes ago   Up 42 minutes
0.0.0.0:3306->3306/tcp, :::3306->3306/tcp, 33060/tcp           ityingMysql
```

```
[root@localhost ~]# docker exec -ti f9b5dd026a4e /bin/bash
root@f9b5dd026a4e:/# mongo
MongoDB shell version v5.0.1
connecting to: mongodb://127.0.0.1:27017/?compressors=disabled&gssapiServiceName=mongodb
Implicit session: session { "id" : UUID("d200ae7f-e85c-4a17-8b24-55f4f116e08a") }
MongoDB server version: 5.0.1
================
Warning: the "mongo" shell has been superseded by "mongosh",
which delivers improved usability and compatibility.The "mongo" shell has been deprecated and will be removed in
an upcoming release.
We recommend you begin using "mongosh".
For installation instructions, see
https://docs.mongodb.com/mongodb-shell/install/
================
Welcome to the MongoDB shell.
For interactive help, type "help".
For more comprehensive documentation, see
        https://docs.mongodb.com/
Questions? Try the MongoDB Developer Community Forums
        https://community.mongodb.com
---
The server generated these startup warnings when booting:
        2021-07-28T04:25:25.178+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
        2021-07-28T04:25:25.178+00:00: /sys/kernel/mm/transparent_hugepage/enabled is 'always'. We suggest setting it to 'never'
---
---
        Enable MongoDB's free cloud-based monitoring service, which will then receive and display
```

```
        metrics about your deployment (disk utilization, CPU, operation statistics, etc).

        The monitoring data will be available on a MongoDB website with a unique URL accessible to you
        and anyone you share the URL with. MongoDB may use this information to make product
        improvements and to suggest MongoDB products and deployment options to you.

        To enable free monitoring, run the following command: db.enableFreeMonitoring()
        To permanently disable this reminder, run the following command: db.disableFreeMonitoring()
---
>
```

远程连接

```
C:\Users\itying>mongo 192.168.241.128:27017
MongoDB shell version v4.2.5
connecting to: mongodb://192.168.241.128:27017/test?compressors=disabled&gssapiServiceName=mongodb
Implicit session: session { "id" : UUID("2e196a91-5657-4dfb-b9e3-c68491385dc1") }
MongoDB server version: 5.0.1
WARNING: shell and server versions do not match
Server has startup warnings:
{"t":{"$date":"2021-07-28T04:25:25.178+00:00"},"s":"W", "c":"CONTROL", "id":22120, "ctx":"initandlisten","msg":"Access control is not enabled for the database. Read and write access to data and configuration is unrestricted","tags":["startupWarnings"]}
{"t":{"$date":"2021-07-28T04:25:25.178+00:00"},"s":"W", "c":"CONTROL", "id":22178, "ctx":"initandlisten","msg":"/sys/kernel/mm/transparent_hugepage/enabled is 'always'. We suggest setting it to 'never'","tags":["startupWarnings"]}
---
Enable MongoDB's free cloud-based monitoring service, which will then receive and display
metrics about your deployment (disk utilization, CPU, operation statistics, etc).

The monitoring data will be available on a MongoDB website with a unique URL accessible to you
and anyone you share the URL with. MongoDB may use this information to make produ
```

```
ct
improvements and to suggest MongoDB products and deployment options to you.

To enable free monitoring, run the following command: db.enableFreeMonitoring()
To permanently disable this reminder, run the following command: db.disableFreeMonitori
ng()
---

> show dbs
```

测试数据持久化

```
> show dbs
admin      0.000GB
config     0.000GB
local      0.000GB
> use itying
switched to db itying
> db.user.insert({username:"张三"})
WriteResult({ "nInserted" : 1 })
> show dbs                                    })
admin      0.000GB
config     0.000GB
itying     0.000GB
local      0.000GB
> exit;
```

```
[root@localhost ~]# docker ps -a
CONTAINER ID     IMAGE        COMMAND                  CREATED              STATUS
            PORTS                                                  NAMES
f9b5dd026a4e     mongo        "docker-entrypoint.s…"   6 minutes ago        Up 6 minutes
 0.0.0.0:27017->27017/tcp, :::27017->27017/tcp                     ityingMongo
83c2eda7329d     redis        "docker-entrypoint.s…"   31 minutes ago       Up 31 minutes
0.0.0.0:6379->6379/tcp, :::6379->6379/tcp                          redis-test
f6dbc4920782     mysql        "docker-entrypoint.s…"   48 minutes ago       Up 48 minutes
0.0.0.0:3306->3306/tcp, :::3306->3306/tcp, 33060/tcp               ityingMysql
```

```
docker run --name ityingMongo -p 27017:27017 -v /root/mongo/data:/data/db -d mongo
```

```
> show dbs
admin    0.000GB
config   0.000GB
itying   0.000GB
local    0.000GB
```

4.1.2、连接需要密码

```
docker run -d --name authMongo \
    -e MONGO_INITDB_ROOT_USERNAME=admin \
    -e MONGO_INITDB_ROOT_PASSWORD=123456 \
    -p 27017:27017 \
    -v /root/mongo/data:/data/db \
    mongo --auth
```

```
[root@localhost ~]# docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED          STATUS
        PORTS                                                  NAMES
1bae12b5f438   mongo      "docker-entrypoint.s…"   3 seconds ago    Up 2 seconds
0.0.0.0:27017->27017/tcp, :::27017->27017/tcp          authMongo
83c2eda7329d   redis      "docker-entrypoint.s…"   3 hours ago      Up 3 hours      0.
0.0.0.0:6379->6379/tcp, :::6379->6379/tcp              redis-test
f6dbc4920782   mysql      "docker-entrypoint.s…"   4 hours ago      Up 4 hours      0.
0.0.0.0:3306->3306/tcp, :::3306->3306/tcp, 33060/tcp   ityingMysql
[root@localhost ~]# docker exec-ti 1bae12b5f438 mongo admin
docker: 'exec-ti' is not a docker command.
See 'docker --help'
```

```
[root@localhost ~]# docker exec -ti 1bae12b5f438 mongo admin
MongoDB shell version v5.0.1
connecting to: mongodb://127.0.0.1:27017/admin?compressors=disabled&gssapiServiceName
=mongodb
Implicit session: session { "id" : UUID("1c53734c-cc5e-41b1-a92c-d413a2a306e4") }
```

```
MongoDB server version: 5.0.1
================
Warning: the "mongo" shell has been superseded by "mongosh",
which delivers improved usability and compatibility.The "mongo" shell has been deprecated and will be removed in
an upcoming release.
We recommend you begin using "mongosh".
For installation instructions, see
https://docs.mongodb.com/mongodb-shell/install/
================
Welcome to the MongoDB shell.
For interactive help, type "help".
For more comprehensive documentation, see
      https://docs.mongodb.com/
Questions? Try the MongoDB Developer Community Forums
      https://community.mongodb.com
> show dbs
> db.auth('admin', '123456')
```

授权

```
db.auth('admin', '123456')
> show dbs
admin     0.000GB
config    0.000GB
itying    0.000GB
local     0.000GB
```

创建一个账户：

```
db.createUser({ user:'zhangsan',pwd:'123456',roles:[ { role:'userAdminAnyDatabase', db: 'admin'},"readWriteAnyDatabase"]});
```

授权

```
> db.createUser({ user:'zhangsan',pwd:'123456',roles:[ { role:'userAdminAnyDatabase', db: 'admin'},"readWriteAnyDatabase"]});
Successfully added user: {
      "user" : "zhangsan",
      "roles" : [
            {
```

```
                "role" : "userAdminAnyDatabase",
                "db" : "admin"
            },
            "readWriteAnyDatabase"
        ]
}
```

远程连接：

注意加上数据库表

```
mongo 192.168.241.128:27017/admin
```

```
C:\Users\htzhanglong>mongo 192.168.241.128:27017/admin
MongoDB shell version v5.0.1
connecting to: mongodb://192.168.241.128:27017/admin?compressors=disabled&gssapiServiceName=mongodb
Implicit session: session { "id" : UUID("c42d2b91-34ac-4d77-9754-38a826fe78b1") }
MongoDB server version: 5.0.1
================
Warning: the "mongo" shell has been superseded by "mongosh",
which delivers improved usability and compatibility.The "mongo" shell has been deprecated and will be removed in
an upcoming release.
We recommend you begin using "mongosh".
For installation instructions, see
https://docs.mongodb.com/mongodb-shell/install/
================
> db.auth("zhangsan","123456")
1
>
```

五、Docker Dockerfile

Dockerfile 就是用来构建 docker 镜像的构建文件，下面就给大家演示一下 Dockerfile 的使用。

5.1、Dockerfile 构建一个自己的 centos 镜像

新建一个名为 Dockerfile 文件，并在文件内添加以下内容：

```
FROM centos
RUN yum install -y net-tools
WORKDIR /home/wwwroot
CMD /bin/bash
```

```
[root@localhost ~]# docker build -t docker.io/itying/centos:v1 .
```

```
[root@localhost ~]# docker images
REPOSITORY          TAG          IMAGE ID          CREATED              SIZE
itying/centos       v1           16a9d3332b5d      54 seconds ago       209MB
alpine              latest       d4ff818577bc      6 weeks ago          5.6MB
hello-world         latest       d1165f221234      4 months ago         13.3kB
centos              latest       300e315adb2f      7 months ago         209MB
```

```
[root@localhost ~]# docker run -ti 884a3e5bb2ae
[root@4ebe03cb2cb8 wwwroot]# pwd
/home/wwwroot
[root@4ebe03cb2cb8 wwwroot]# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.17.0.2  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 02:42:ac:11:00:02  txqueuelen 0  (Ethernet)
        RX packets 8  bytes 648 (648.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

5.2、Dockerfile 构建一个 nginx 镜像

Dockerfile 构建一个 nginx 镜像,构建好的镜像内会有一个 /usr/share/nginx/html/index.html 文件

新建一个名为 Dockerfile 文件，并在文件内添加以下内容：

```
FROM nginx
RUN echo '你好 docker' > /usr/share/nginx/html/index.html
WORKDIR /usr/share/nginx/html
```

注意编译的时候的最后的点

```
[root@localhost ~]# vi Dockerfile_nginx
[root@localhost ~]# docker build -t docker.io/itying/nginx:v1 .
```

```
[root@localhost ~]# docker images
REPOSITORY                  TAG       IMAGE ID       CREATED              SIZE
itying/nginx                v1        bc98e0953a89   26 seconds ago       133MB
itying/centos               v1        884a3e5bb2ae   12 minutes ago       249MB
mongo                       latest    49f9af59c1e4   30 hours ago         682MB
itying/alpine_aaa_text      v1.0.1    50cd88a377bf   3 days ago           5.6MB
```

```
docker run -ti -d -p 80:80 bc98e0953a89


[root@localhost ~]# curl 127.0.0.1

你好 docker
```

这是一个本地构建的 nginx 镜像

5.3、Dockerfile 详解

注意：

1、Dockerfile 文件的文件名建议使用 Dockerfile ，如果是其他文件构建的时候需要指定文件名

2、Dockerfile 构建镜像的执行顺序是从上往下

3、#表示注释

4、每一个指令都会创建一个新的镜像层，并提交

```
FROM            # 基础境像,一切从这里开始构建
MAINTAINER      # 镜像是谁写的,姓名+邮箱
LABEL           # LABEL 指令用来给镜像添加一些元数据
RUN             # 编译镜像时运行的脚本
COPY            # 编译镜像时复制文件到镜像中    不会减压
ADD             # 编译镜像时复制文件到镜像中    tar.gz 文件会自动减压
WORKDIR         # 镜像的工作目录
CMD             # 设置容器启动的命令
ENTRYPOINT      # 设置容器启动的命令
EXPOSE          # 设置镜像暴露的端口
VOLUME          # 设置容器挂载的卷
ENV             # 设置容器的环境变量
```

FROM

指定哪种镜像作为新镜像的基础镜像，如：

```
FROM  ubuntu:14.04
```

MAINTAINER

指明该镜像的作者和其电子邮件，如：

```
MAINTAINER  "xxxxxxx@qq.com"
```

LABEL

给镜像添加信息。使用 docker inspect 可查看镜像的相关信息

```
LABEL  maintainer="itying@qq.com"
LABEL  version="1.0"
LABEL  description="This  is  description"
```

RUN

在新镜像内部执行的命令，比如安装一些软件、配置一些基础环境，可使用\来换行，如：

```
RUN  echo  'hello  docker!'  \
     >  /usr/local/file.txt
RUN  yum  install  net-tools  -y
```

也可以使用 exec 格式 RUN ["executable", "param1", "param2"]的命令，如：

```
RUN  ["apt-get","install","-y","nginx"]
RUN  ["yum","install","-y","nginx"]
```

COPY

将主机的文件复制到镜像内，如果目的位置不存在，Docker 会自动创建所有需要的目录结构，但是它只是单纯的复制，并不会去做文件提取和解压工作。如：

```
COPY  application.yml  /etc/springboot/hello-service/src/resources
```

ADD

将主机的文件复制到镜像中，跟 COPY 一样，限制条件和使用方式都一样，如：

```
ADD  application.yml  /etc/springboot/hello-service/src/resources
```

但是 ADD 会对压缩文件（tar, gzip, bzip2, etc）做提取和解压操作。

WORKDIR

在构建镜像时，指定镜像的工作目录，之后的命令都是基于此工作目录，如果不存在，则会创建目录。如下：最终会在/usr/local/webservice/目录下生成 text.txt 文件

```
WORKDIR  /usr/local
WORKDIR  webservice
RUN  echo  'hello  docker'  >  text.txt
```

用户切换目录 类似 cd 命令

```
WORKDIR /root/code
RUN npm run start
```

CMD

容器启动时需要执行的命令，如：

```
CMD /bin/bash
```

同样可以使用 exec 语法，如

```
CMD ["/bin/bash"]
```

ENTRYPOINT

CMD 和 ENTRYPOINT 同样作为容器启动时执行的命令，区别有以下几点：

- CMD 的命令会被 docker run 的命令覆盖而 ENTRYPOINT 不会

如使用 CMD ["/bin/bash"]或 ENTRYPOINT ["/bin/bash"]后，再使用 docker run -ti image 启动容器，它会自动进入容器内部的交互终端，如同使用
docker run -ti image /bin/bash。

但是如果启动镜像的命令为 docker run -ti image /bin/ps，使用 CMD 后面的命令就会被覆盖转而执行 bin/ps 命令，而 *ENTRYPOINT 的则不会，而是会把 docker run 后面的命令当做 ENTRYPOINT 执行命令的参数*。
以下例子比较容易理解
Dockerfile 中为

...

```
ENTRYPOINT ["/user/sbin/nginx"]
```

EXPOSE 暴露端口

仅仅只是声明端口。

作用：

- 帮助镜像使用者理解这个镜像服务的守护端口，以方便配置映射。
- 在运行时使用随机端口映射时，也就是 docker run -P 时，会自动随机映射 EXPOSE 的

端口。

在运行时使用随机端口映射时，也就是 docker run -P 时，会自动随机映射 EXPOSE 的端口。

```
EXPOSE 8080
EXPOSE 8081
```

VOLUME

通过 dockerfile 的 VOLUME 指令可以在镜像中创建挂载点，这样只要通过该镜像创建的容器都有了挂载点。

注意：通过 VOLUME 指令创建的挂载点，无法指定主机上对应的目录，是自动生成的。

格式：

```
VOLUME ["<路径 1>", "<路径 2>"...]
VOLUME <路径>
```

我们通过 docker inspect 查看通过该 dockerfile 创建的镜像生成的容器

ENV

设置环境变量，定义了环境变量，那么在后续的指令中，就可以使用这个环境变量。

格式：

```
ENV <key> <value>
ENV <key1>=<value1> <key2>=<value2>...
```

以下示例设置 NODE*VERSION = 7.2.0* ， *在后续的指令中可以通过 $NODE*VERSION 引用：

```
ENV NODE_VERSION 7.2.0

RUN curl -SLO "https://nodejs.org/dist/v$NODE_VERSION/node-v$NODE_VERSION-linux-x64.tar.xz" \
  && curl -SLO "https://nodejs.org/dist/v$NODE_VERSION/SHASUMS256.txt.asc"
```

ONBUILD

用于延迟构建命令的执行。简单的说，就是 Dockerfile 里用 ONBUILD 指定的命令，在本次构建镜像的过程中不会执行（假设镜像为 test-build）。当有新的 Dockerfile 使用了之前构建的镜像 FROM test-build ，这时执行新镜像的 Dockerfile 构建时候，会执行 test-build 的 Dockerfile 里的 ONBUILD 指定的命令。

格式：

```
ONBUILD <其它指令>
```

5.4、Dockerfile 构建 Centos 并安装 net-tools yum 软件

新建 Dockerfile_Centos

```
FROM centos
MAINTAINER itying.com
ENV MyLocal /usr/local
WORKDIR $MyLocal
EXPOSE 80
VOLUME ["volume1","volume2"]
RUN yum install -y net-tools
RUN yum install -y vim
ADD test.tar.gz /root
COPY test.tar.gz /usr/local
CMD /bin/bash
```

编译 编译的时候注意最后面的.

```
docker build -f Dockerfile_centos -t ityingcentos:v1.0 .
```

```
[root@localhost ~]# docker build -f Dockerfile_centos -t ityingcentos:v1.0 .
```

```
[root@localhost ~]# docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
ityingcentos v1.0 39ddb6dcc84b 23 seconds ago 302MB
centos latest 300e315adb2f 7 months ago 209MB
```

查看执行的历史

```
[root@localhost ~]# docker history 39ddb6dcc84b
IMAGE               CREATED             CREATED BY
    SIZE          COMMENT
39ddb6dcc84b    15 minutes ago    /bin/sh -c #(nop)   CMD ["/bin/sh" "-c" "/bin…    0B

bd28224c0f8c    15 minutes ago    /bin/sh -c #(nop)   CMD ["/bin/sh" "-c" "echo…    0B

48f53d896afc    15 minutes ago    /bin/sh -c #(nop)   CMD ["/bin/sh" "-c" "echo…    0B

e7f9d6cc030c    15 minutes ago    /bin/sh -c #(nop) COPY file:0da3969080c35251…    159
B
284656edc586    15 minutes ago    /bin/sh -c #(nop) ADD file:0da3969080c35251f…    0B

bf1888b898f3    15 minutes ago    /bin/sh -c yum install -y vim                    52.6
MB
f537a1bc707f    15 minutes ago    /bin/sh -c yum install -y net-tools              39.6M
B
abe00ab0fe7a    15 minutes ago    /bin/sh -c #(nop)   VOLUME [/root/wwwroot /ro…    0
B
d3368481e833    15 minutes ago    /bin/sh -c #(nop)   EXPOSE 80                     0
B
d293e9670bb8    15 minutes ago    /bin/sh -c #(nop) WORKDIR /usr/local              0B

20b22186738f    15 minutes ago    /bin/sh -c #(nop)   ENV MyLocal=/usr/local        0B

68558121ce4f    15 minutes ago    /bin/sh -c #(nop)   MAINTAINER itying.com        0B

300e315adb2f    7 months ago      /bin/sh -c #(nop)   CMD ["/bin/bash"]            0B

<missing>       7 months ago      /bin/sh -c #(nop)   LABEL org.label-schema.sc…    0B

<missing>       7 months ago      /bin/sh -c #(nop) ADD file:bd7a2aed6ede423b7…    20
9MB
```

执行了 CMD 中配置的 /bin/bash 并且进入后进入了 WORKDIR 目录

```
[root@localhost ~]# docker run -it 39ddb6dcc84b
[root@07d971fca48a local]#
```

RUN yum install -y net-tools 中的软件已安装

```
[root@07d971fca48a local]# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.17.0.2  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 02:42:ac:11:00:02  txqueuelen 0  (Ethernet)
        RX packets 11  bytes 906 (906.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

root 中进行了目录映射，并且执行了 ADD test.tar.gz /root 把 test.tar.gz 减压到了 root 目录

```
[root@07d971fca48a local]# ls /root/
anaconda-ks.cfg  anaconda-post.log  original-ks.cfg  test  wwwroot
```

COPY test.tar.gz /usr/local 查看 usr/local/目录

```
[root@07d971fca48a /]# ls usr/local/
bin  etc  games  include  lib  lib64  libexec  sbin  share  src  test.tar.gz
```

5.5、Docker CMD 和 ENTRYPOINT 对比

CMD 和 ENTRYPOINT 同样作为容器启动时执行的命令，区别有以下几点：

- CMD 的命令会被 docker run 的命令覆盖而 ENTRYPOINT 不会

如使用 CMD ["/bin/bash"]或 ENTRYPOINT ["/bin/bash"]后，再使用 docker run -ti image 启动容器，它会自动进入容器内部的交互终端，如同使用
docker run -ti image /bin/bash。

但是如果启动镜像的命令为 docker run -ti image /bin/ps，使用 CMD 后面的命令就会被覆盖转而执行 bin/ps 命令，而 *ENTRYPOINT 的则不会，而是会把 docker run 后面的命令当做*

ENTRYPOINT 执行命令的参数。

以下例子比较容易理解

Dockerfile 中为

```
...
ENTRYPOINT  ["/user/sbin/nginx"]
```

Docker CMD

新建 Dockerfile_text

```
FROM  centos
CMD  "/bin/bash"
[root@localhost  ~]#  docker  build  -f  Dockerfile_text  -t  cmdtext:v1.0.1  .
Sending  build  context  to  Docker  daemon   234.4MB
Step  1/2  :  FROM  centos
 --->  300e315adb2f
Step  2/2  :  CMD  "/bin/bash"
 --->  Running  in  91127ebc2da9
Removing  intermediate  container  91127ebc2da9
 --->  6dd84e5dc8ec
Successfully  built  6dd84e5dc8ec
Successfully  tagged  cmdtext:v1.0.1
[root@localhost  ~]#  docker  images
REPOSITORY     TAG           IMAGE  ID          CREATED              SIZE
cmdtext         v1.0.1       6dd84e5dc8ec     20  seconds  ago     209MB
centos          latest       300e315adb2f     7  months  ago       209MB
[root@localhost  ~]#  docker  run  -it  6dd84e5dc8ec
[root@29237ee377ae  /]#  exit
exit
```

追加一个命令

```
[root@localhost  ~]#  docker  run  -it  6dd84e5dc8ec  /bin/sh
sh-4.4#
```

Docker ENTRYPOINT

新建 Dockerfile_text

```
FROM  centos
```

```
ENTRYPOINT "/bin/bash"
```

```
[root@localhost ~]# docker build -f Dockerfile_text -t entrypointtext:v1.0.1 .
Sending build context to Docker daemon  234.4MB
Step 1/2 : FROM centos
 ---> 300e315adb2f
Step 2/2 : ENTRYPOINT "/bin/bash"
 ---> Running in 6999b763dea6
Removing intermediate container 6999b763dea6
 ---> 64d65b5e8e48
Successfully built 64d65b5e8e48
Successfully tagged entrypointtext:v1.0.1
[root@localhost ~]# docker images
REPOSITORY          TAG          IMAGE ID          CREATED             SIZE
entrypointtext      v1.0.1       64d65b5e8e48      4 seconds ago       209MB
cmdtext             v1.0.1       6dd84e5dc8ec      2 minutes ago       209MB
centos              latest       300e315adb2f      7 months ago        209MB
```

```
[root@localhost ~]# docker run -it 6dd84e5dc8ec
[root@78a0632d7810 /]#
```

```
[root@localhost ~]# docker run -it 6dd84e5dc8ec /bin/sh
[root@b0f01f4ae923 /]#
```

六、Dockerfile 自动部署 Nodejs 程序

项目目录中新建 Dockerfile

COPY . /root/wwwroot/表示把项目目录中的代码复制到容器里面的/root/wwwroot 目录

```
FROM node
COPY . /root/wwwroot/
WORKDIR /root/wwwroot/
EXPOSE 3000
RUN npm install cnpm -g --registry=https://registry.nlark.com
RUN cnpm install
CMD node app.js
```

```
[root@localhost ~]# docker build -t nodeimg:v1.0.1 .
```

```
[root@localhost ~]# docker images
REPOSITORY      TAG          IMAGE ID          CREATED                SIZE
nodeimg         v1.0.1       224c3a3afd01      31 seconds ago         968MB
ityingnginx     v1.0.1       355d427cd372      About an hour ago      133MB
node            latest       8f1b7f0dfc2f      6 days ago             907MB
nginx           latest       08b152afcfae      6 days ago             133MB
centos          latest       300e315adb2f      7 months ago           209MB
```

```
[root@localhost ~]# docker run -tid --name node02  -p 3000:3000 nodeimg:v1.0.1
044fbd812d026c6ded002516c2b21e0dcb27955156294c6376179c1518bce95c
```

```
[root@localhost ~]# docker ps
CONTAINER ID    IMAGE                COMMAND                  CREATED             ST
ATUS            PORTS                                                NAMES
044fbd812d02    nodeimg:v1.0.1    "docker-entrypoint.s…"    3 seconds ago    Up 1 second
    0.0.0.0:3000->3000/tcp, :::3000->3000/tcp    node02
```

http://192.168.241.128:3000/

七、配置 Docker Dockerfile 部署 Golang

7.1、回顾一下 golang beego 打包以及部署

windows：

filename： 文件名

```
set GOOS=windows
set GOARCH=amd64
go build -o "filename"
```

直接双击 filename.exe 文件执行即可

linux：

```
set  GOOS=linux
set  GOARCH=amd64
go  build  -o  "filename"
```

上传到 linux 中赋予执行权限

```
chmod  -R  777  目录
```

项目执行

```
nohup  ./filename  &
```

beego 项目打包

```
bee  pack

bee  pack  -be  GOOS=window

bee  pack  -be  GOOS=linux
```

上传到 linux 中赋予执行权限

```
chmod  -R  777  目录
```

运行项目：

```
nohup  ./beepkg  &
```

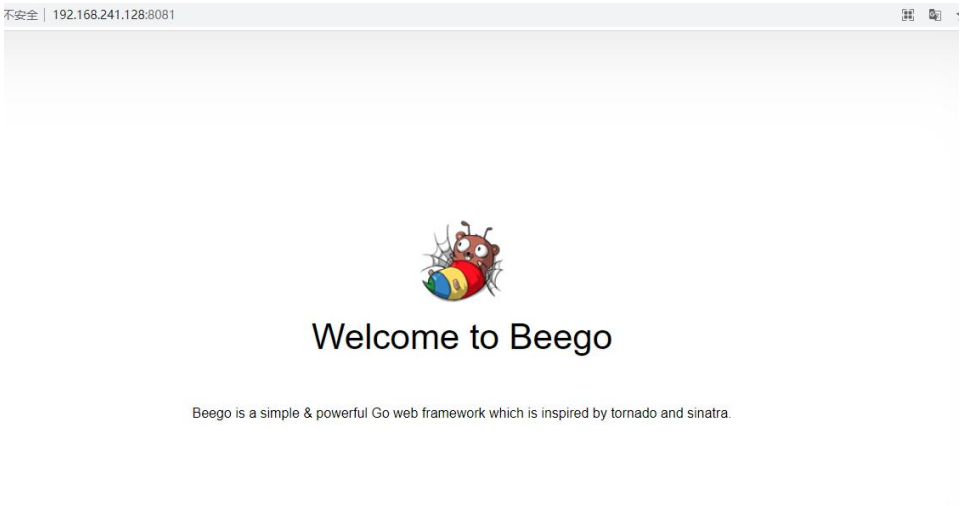
7.2、Docker 部署编译好的 golang 项目

```
FROM centos
COPY . /root/golang
WORKDIR /root/golang
EXPOSE 8080
ENTRYPOINT ["./beegodemo01"]
```

```
[root@localhost beegodemo]# docker build -t golangimg:v1.0.1 .
Sending build context to Docker daemon  14.62MB
Step 1/5 : FROM centos
 ---> 300e315adb2f
Step 2/5 : COPY . /root/golang
 ---> 78670b1c00eb
Step 3/5 : WORKDIR /root/golang
 ---> Running in 850af8309c34
Removing intermediate container 850af8309c34
 ---> f6f6f3ef99ed
Step 4/5 : EXPOSE 8080
 ---> Running in f1ea11c5c19b
Removing intermediate container f1ea11c5c19b
 ---> ba11828a3ef1
Step 5/5 : ENTRYPOINT ["./beegodemo01"]
 ---> Running in 57d921e41e48
Removing intermediate container 57d921e41e48
 ---> 87700337cda3
Successfully built 87700337cda3
Successfully tagged golangimg:v1.0.1
```

```
[root@localhost ~]# docker images
REPOSITORY      TAG          IMAGE ID          CREATED              SIZE
golangimg       v1.0.1       87700337cda3      10 minutes ago       224MB
nodeimg         v2.0.1       f6592d99a245      17 hours ago         968MB
```

```
[root@localhost ~]# docker run -tid -p 8081:8080 golangimg:v1.0.1
a38eb4ed369053f2d6c1209fb9070e388616613916f5dc228f5c539efd3aaeee
```



注意：如果映射端口失败的话可以重启 docker

```
systemctl restart docker
```

7.3、Docker 部署未编译的 golang 项目

1、goweb 目录新建 main.go

```
package main
import (
    "fmt"
    "net/http"
)
func handlerHello(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "hello docker")
}
func main() {
    http.HandleFunc("/", handlerHello)
    http.ListenAndServe(":8080", nil)
}
```

```
FROM golang
MAINTAINER "itying@qq.com"
ADD . /root/goweb
WORKDIR /root/goweb
RUN go build
EXPOSE 8080
ENTRYPOINT ["./goweb"]
```

2、在工程目录 goweb 下，新建 dockerfile 文件

```
[root@localhost goweb]# docker build -t golangimg:v2.0.1 .
Sending build context to Docker daemon  4.096kB
Step 1/7 : FROM golang
latest: Pulling from library/golang
627b765e08d1: Already exists
c040670e5e55: Already exists
073a180f4992: Already exists
bf76209566d0: Already exists
6182a456504b: Pull complete
cc29a4876382: Pull complete
ff36ba465698: Pull complete
Digest: sha256:4544ae57fc735d7e415603d194d9fb09589b8ad7acd4d66e928eabfb1ed85ff1
Status: Downloaded newer image for golang:latest
```

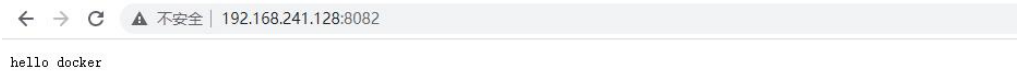
```
[root@localhost goweb]# docker images
REPOSITORY      TAG         IMAGE ID        CREATED                 SIZE
golangimg       v2.0.1      64ec7f53cb96    About a minute ago      868MB
golangimg       v1.0.1      87700337cda3    39 minutes ago          224MB
```

```
docker run -tid -p 8082:8080 golangimg:v2.0.1
```

```
[root@localhost goweb]# docker ps
CONTAINER ID    IMAGE                   COMMAND             CREATED             STATUS
            PORTS                                           NAMES
```

```
ea59aa548ba0    golangimg:v2.0.1    "./goweb"              26 seconds ago    Up 25 seconds
    0.0.0.0:8082->8080/tcp, :::8082->8080/tcp    confident_stonebraker
a38eb4ed3690    golangimg:v1.0.1    "./beegodemo01"    30 minutes ago    Up 30 minute
s    0.0.0.0:8081->8080/tcp, :::8081->8080/tcp    affectionate_black
```

```
[root@localhost goweb]# docker exec -ti ea59aa548ba0 /bin/bash
root@ea59aa548ba0:~/goweb# curl 127.0.0.1:8080
```



7.4、Docker 部署未编译的 beego 项目

需要配置环境变量

```
$ export GO111MODULE=on
$ export GOPROXY=https://goproxy.cn
```

配置 beegodemo01 项目中配置 Dockerfile

```
FROM golang
MAINTAINER "itying@qq.com"
ADD . /root/beegodemo01
WORKDIR /root/beegodemo01
ENV GO111MODULE=on
ENV GOPROXY=https://goproxy.cn
RUN go get github.com/beego/bee
EXPOSE 8080
ENTRYPOINT ["bee", "run"]
```

```
[root@localhost beegodemo01]# docker build -t golangimg:v3.0.1 .
```

```
Sending build context to Docker daemon  92.67kB
Step 1/9 : FROM golang
 ---> 028d102f774a
Step 2/9 : MAINTAINER "itying@qq.com"
 ---> Using cache
 ---> 41c15ed577fb
Step 3/9 : ADD . /root/beegodemo01
 ---> b5b8b28d6b2a
Step 4/9 : WORKDIR /root/beegodemo01
 ---> Running in 14e816e25491
Removing intermediate container 14e816e25491
 ---> aec0d306684e
Step 5/9 : ENV GO111MODULE=on
 ---> Running in 5b6e0b9ef84e
Removing intermediate container 5b6e0b9ef84e
 ---> ce6985607bd9
Step 6/9 : ENV GOPROXY=https://goproxy.cn
 ---> Running in 3f38275ccc3a
Removing intermediate container 3f38275ccc3a
 ---> 97cfb34173bf
Step 7/9 : RUN go get github.com/beego/bee
 ---> Running in 7835471d1ddb
go: downloading github.com/beego/bee v1.12.3
go: downloading gopkg.in/yaml.v2 v2.3.0
go: downloading github.com/fsnotify/fsnotify v1.4.9
go: downloading github.com/gadelkareem/delve v1.4.2-0.20200619175259-dcd01330766f
go: downloading github.com/gorilla/websocket v1.4.2
go: downloading github.com/go-sql-driver/mysql v1.5.0
go: downloading github.com/lib/pq v1.7.0
go: downloading github.com/davecgh/go-spew v1.1.1
go: downloading github.com/flosch/pongo2 v0.0.0-20200529170236-5abacdfa4915
go: downloading github.com/pelletier/go-toml v1.2.0
go: downloading github.com/smartwalle/pongo2render v1.0.1
go: downloading github.com/spf13/viper v1.7.0
go: downloading golang.org/x/sys v0.0.0-20191005200804-aed5e4c7ecf9
go: downloading github.com/cosiner/argv v0.1.0
go: downloading github.com/mattn/go-colorable v0.0.9
go: downloading github.com/peterh/liner v0.0.0-20170317030525-88609521dc4b
go: downloading github.com/sirupsen/logrus v1.6.0
go: downloading github.com/astaxie/beego v1.12.1
go: downloading github.com/hashicorp/hcl v1.0.0
go: downloading github.com/magiconair/properties v1.8.1
go: downloading github.com/mitchellh/mapstructure v1.1.2
```

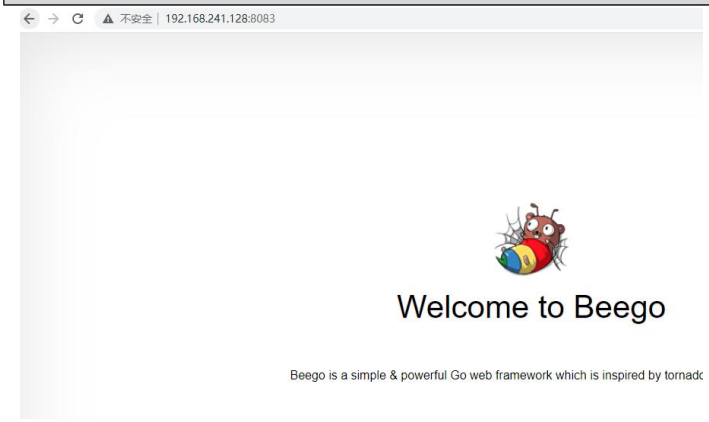
```
go: downloading github.com/spf13/afero v1.1.2
go: downloading github.com/spf13/cast v1.3.0
go: downloading github.com/spf13/jwalterweatherman v1.0.0
go: downloading github.com/spf13/pflag v1.0.3
go: downloading github.com/subosito/gotenv v1.2.0
go: downloading gopkg.in/ini.v1 v1.51.0
go: downloading go.starlark.net v0.0.0-20190702223751-32f345186213
go: downloading github.com/mattn/go-isatty v0.0.3
go: downloading github.com/hashicorp/golang-lru v0.5.4
go: downloading golang.org/x/arch v0.0.0-20190927153633-4e8777c89be4
go: downloading github.com/konsorten/go-windows-terminal-sequences v1.0.3
go: downloading golang.org/x/text v0.3.2
go get: added github.com/beego/bee v1.12.3
Removing intermediate container 7835471d1ddb
 ---> 2255e06dbfdc
Step 8/9 : EXPOSE 8080
 ---> Running in 756c347ba199
Removing intermediate container 756c347ba199
 ---> 389820f8f6b5
Step 9/9 : ENTRYPOINT bee run
 ---> Running in 3fa8d8cce7b4
Removing intermediate container 3fa8d8cce7b4
 ---> 931a238bfbbb
Successfully built 931a238bfbbb
Successfully tagged golangimg:v3.0.1
```

```
[root@localhost beegodemo01]# docker images
REPOSITORY      TAG         IMAGE ID        CREATED                 SIZE
golangimg       v3.0.1      931a238bfbbb    About a minute ago      1.01GB
golangimg       v2.0.1      64ec7f53cb96    42 minutes ago          868MB
golangimg       v1.0.1      87700337cda3    About an hour ago       224MB
nodeimg         v2.0.1      f6592d99a245    18 hours ago            968MB
nodeimg         v1.0.1      224c3a3afd01    19 hours ago            968MB
ityingnginx     v1.0.1      355d427cd372    20 hours ago            133MB
```

```
docker run -tid -p 8083:8080 --name beegodemo golangimg:v3.0.1
```

```
[root@localhost beegodemo01]# docker ps
CONTAINER ID    IMAGE                   COMMAND                     CREATED
```

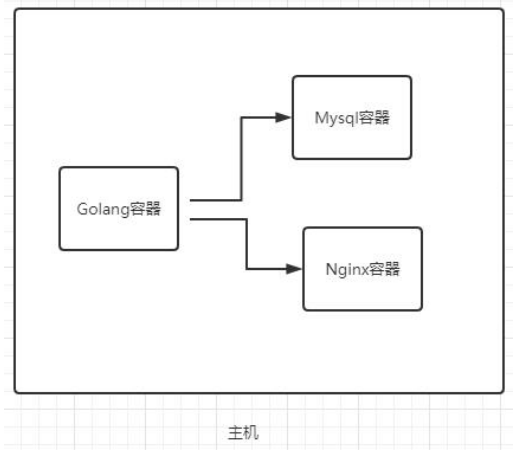
```
      STATUS                  PORTS                                                  NAMES
1c96e85d3c03   golangimg:v3.0.1   "/bin/sh -c 'bee run'"   7 seconds ago        Up 6 s
econds          0.0.0.0:8083->8080/tcp, :::8083->8080/tcp     beegodemo
ea59aa548ba0   golangimg:v2.0.1   "./goweb"                40 minutes ago       Up
40 minutes      0.0.0.0:8082->8080/tcp, :::8082->8080/tcp     confident_stonebraker
a38eb4ed3690   golangimg:v1.0.1   "./beegodemo01"          About an hour ago    Up
 About an hour   0.0.0.0:8081->8080/tcp, :::8081->8080/tcp     affectionate_black

[root@localhost beegodemo01]# docker exec -ti 1c96e85d3c03 /bin/bash
root@1c96e85d3c03:~/beegodemo01# curl 127.0.0.1:8080
```



八、配置 Docker 网络

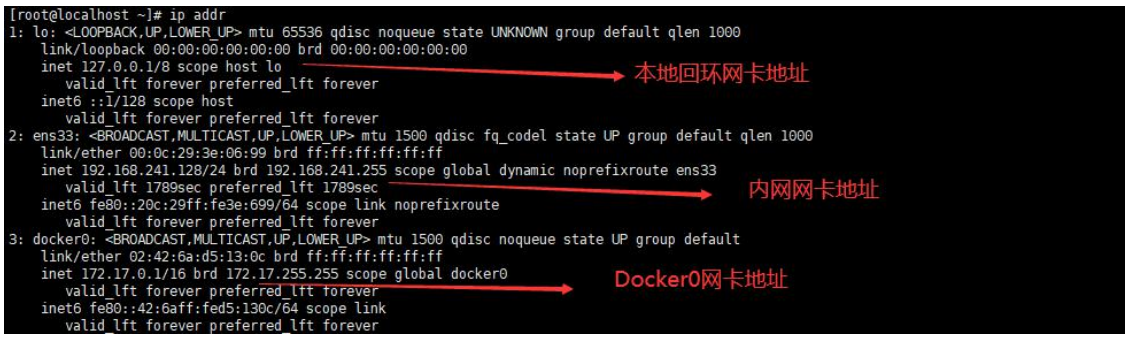
8.1、Docker0 网络

8.1.1、思考：多个容器之间如何通信，是否可以直接连接



首先看看网卡信息

```
ip addr
```




```
[root@localhost ~]# docker images
REPOSITORY      TAG         IMAGE ID        CREATED          SIZE
node            latest      8f1b7f0dfc2f    7 days ago       907MB
nginx           latest      08b152afcfae    7 days ago       133MB
centos          latest      300e315adb2f    7 months ago     209MB
```

启动三个容器测试网络的互通

```
[root@localhost ~]# docker run -tid --name centos01 centos /bin/bash
ae40e0e5f24cadb0f5bf27285817f59eb680a23c1e1272c9ef5a147d26926823

[root@localhost ~]# docker run -tid --name centos02 centos /bin/bash
91a96251fda3de7a575cad1a45e21433bff651b73b242d7592edda3973c73c3e

[root@localhost ~]# docker run -tid --name centos03 centos /bin/bash
cc92713e5a876ed122ac49a9bf5644ec79b885185763abcc1de2d22d0fd9fe4f
```

```
[root@localhost ~]# docker ps
CONTAINER ID    IMAGE       COMMAND          CREATED              STATUS              POR
TS      NAMES
cc92713e5a87    centos      "/bin/bash"      8 seconds ago        Up 7 seconds
centos03
91a96251fda3    centos      "/bin/bash"      17 seconds ago       Up 16 seconds
centos02
ae40e0e5f24c    centos      "/bin/bash"      21 seconds ago       Up 20 seconds
centos01
```

分别查看 3 块网卡的 IP 信息

```
[root@localhost ~]# docker exec -ti centos01 ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
148: eth0@if149: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:ac:11:00:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
       valid_lft forever preferred_lft forever


[root@localhost ~]# docker exec -ti centos02 ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
150: eth0@if151: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:ac:11:00:03 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.17.0.3/16 brd 172.17.255.255 scope global eth0
       valid_lft forever preferred_lft forever

[root@localhost ~]# docker exec -ti centos03 ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
152: eth0@if153: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:ac:11:00:04 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.17.0.4/16 brd 172.17.255.255 scope global eth0
       valid_lft forever preferred_lft forever
```

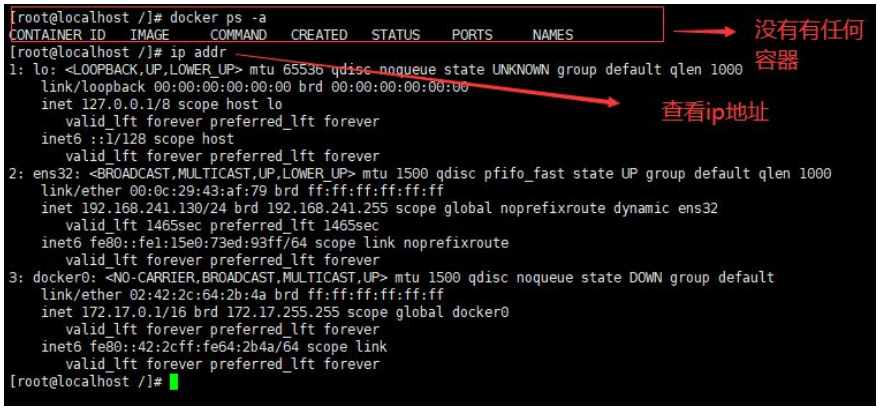
```
[root@localhost ~]# docker exec -ti centos03 /bin/bash
```

```
[root@cc92713e5a87 /]# ping 172.17.0.2
PING 172.17.0.2 (172.17.0.2) 56(84) bytes of data.
64 bytes from 172.17.0.2: icmp_seq=1 ttl=64 time=0.138 ms
64 bytes from 172.17.0.2: icmp_seq=2 ttl=64 time=0.060 ms
64 bytes from 172.17.0.2: icmp_seq=3 ttl=64 time=0.076 ms
64 bytes from 172.17.0.2: icmp_seq=4 ttl=64 time=0.065 ms
```

结论：默认情况同一台主机上面的容器是可以互相通信的，默认情况同一台主机上面的容器和主机之间是可以互相通信的。

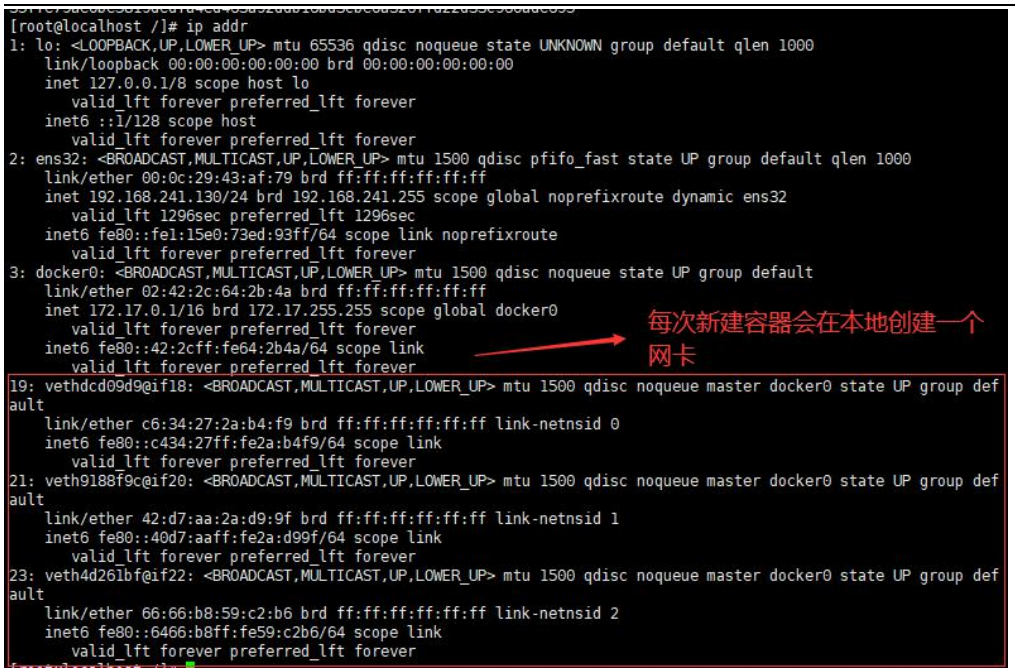
8.1.2 通信原理

我们每启动一个 Docker 容器，Docker 就会给 Docker 容器分配一个 ip，我们只要安装了 Docker，就会有一个网卡 Docker0， Docker0 使用的是桥接模式，使用的技术是 evth-pair 技术。



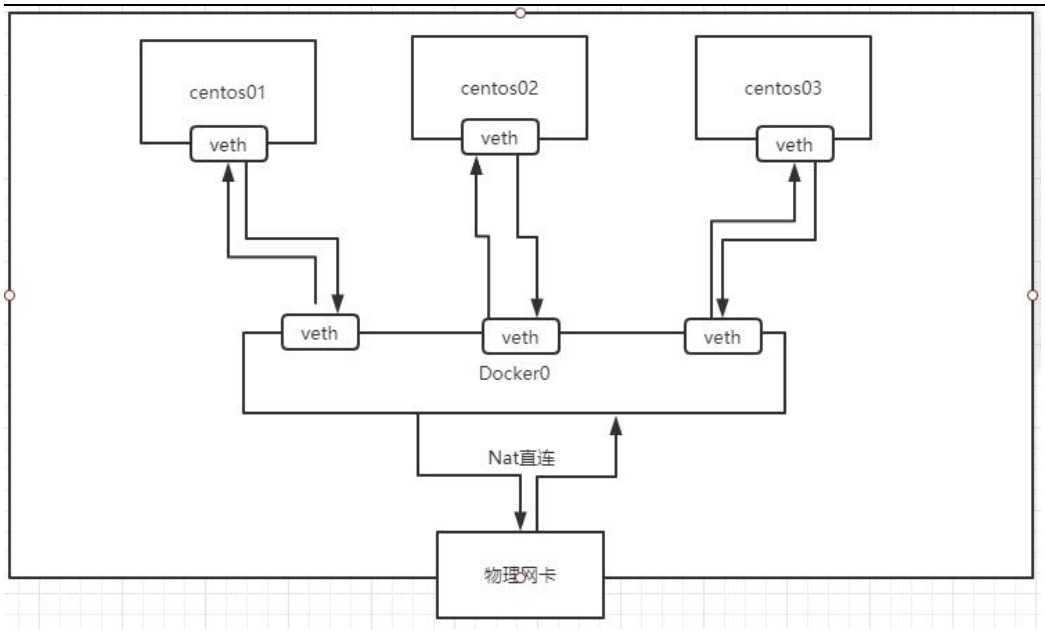
重新创建三个容器：

```
[root@localhost /]# docker run -tid --name centos01 centos /bin/bash
4b602f76e865d71e2184c356f87bc9ab22a1b6c54cc945efd1463d1fcd105266
[root@localhost /]# docker run -tid --name centos02 centos /bin/bash
e69867b51c8c787a296e118484b941621963fc8ba987d2ee4aba67101c654176
[root@localhost /]# docker run -tid --name centos03 centos /bin/bash
35ffe79ae0be5819dedfa4ed463a92ddb16bd3cbe0a326ffd22d33c960ade695
```




```
[root@localhost /]# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
NAMES
35ffe79ae0be centos "/bin/bash" 2 minutes ago Up 2 minutes ce
ntos03
e69867b51c8c centos "/bin/bash" 2 minutes ago Up 2 minutes ce
ntos02
4b602f76e865 centos "/bin/bash" 2 minutes ago Up 2 minutes ce
ntos01
```

```
[root@localhost /]# docker exec -ti centos01 /bin/bash
[root@4b602f76e865 /]# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defa
ult qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
18: eth0@if19: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
 group default
    link/ether 02:42:ac:11:00:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
       valid_lft forever preferred_lft forever
```



8.1.3、使用默认网络的问题

1、没法实现网络隔离

2、没法使用计算机主机名实现通信

8.2、Docker Network 详解

8.2.1、关于 docker network 命令

```
[root@localhost /]# docker network --help

Usage:  docker network COMMAND

Manage networks

Commands:
  connect     Connect a container to a network
  create      Create a network
  disconnect  Disconnect a container from a network
  inspect     Display detailed information on one or more networks
  ls          List networks
  prune       Remove all unused networks
  rm          Remove one or more networks
```

```
Run 'docker network COMMAND --help' for more information on a command
```

8.2.2、docker network ls 查看网络

```
[root@localhost /]# docker network ls
NETWORK ID          NAME          DRIVER       SCOPE
fd4b5495e02e      bridge        bridge       local
c942dbad2529      host          host          local
3d2cedfba066      none          null          local
```

8.2.3、docker network inspect 查看网络详情

```
[root@localhost /]# docker network ls
NETWORK ID          NAME          DRIVER       SCOPE
fd4b5495e02e      bridge        bridge       local
c942dbad2529      host          host          local
3d2cedfba066      none          null          local
```

```
[root@localhost /]# docker network inspect fd4b5495e02e
[
    {
        "Name": "bridge",
        "Id": "fd4b5495e02e8cb7ee91ffc31c73e7da296c440d3bc0640fe342c6445a4ef041",
        "Created": "2021-08-10T15:43:37.562320107+08:00",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.17.0.0/16",
                    "Gateway": "172.17.0.1"
                }
            ]
        },
```

```
            "Internal": false,
            "Attachable": false,
            "Ingress": false,
            "ConfigFrom": {
                "Network": ""
            },
            "ConfigOnly": false,
            "Containers": {
                "35ffe79ae0be5819dedfa4ed463a92ddb16bd3cbe0a326ffd22d33c960ade695": {
                    "Name": "centos03",
                    "EndpointID": "c81bc51f565d478964129d18b1dd9f994209a63d62fda41b21
1d60fb407aee7c",
                    "MacAddress": "02:42:ac:11:00:04",
                    "IPv4Address": "172.17.0.4/16",
},
                "4b602f76e865d71e2184c356f87bc9ab22a1b6c54cc945efd1463d1fcd105266": {
                    "Name": "centos01",
                    "EndpointID": "5e7743d4c9a084adb046dd950b64ea7207eec06aabbe9e9f1a
70216cf0ca43b9",
                    "MacAddress": "02:42:ac:11:00:02",
                    "IPv4Address": "172.17.0.2/16",
                    "IPv6Address": ""
                },
                "e69867b51c8c787a296e118484b941621963fc8ba987d2ee4aba67101c654176":
{
                    "Name": "centos02",
                    "EndpointID": "6a61af374e036de42d18b59ead9d3f3c69b05867b9436d4409
f359f6a5fa2d68",
                    "MacAddress": "02:42:ac:11:00:03",
                    "IPv4Address": "172.17.0.3/16",
                    "IPv6Address": ""
                }
            },
            "Options": {
                "com.docker.network.bridge.default_bridge": "true",
                "com.docker.network.bridge.enable_icc": "true",
                "com.docker.network.bridge.enable_ip_masquerade": "true",
                "com.docker.network.bridge.host_binding_ipv4": "0.0.0.0",
                "com.docker.network.bridge.name": "docker0",
                "com.docker.network.driver.mtu": "1500"
            },
            "Labels": {}
        }
```

]

8.2.4、Docker 网络的四种模式

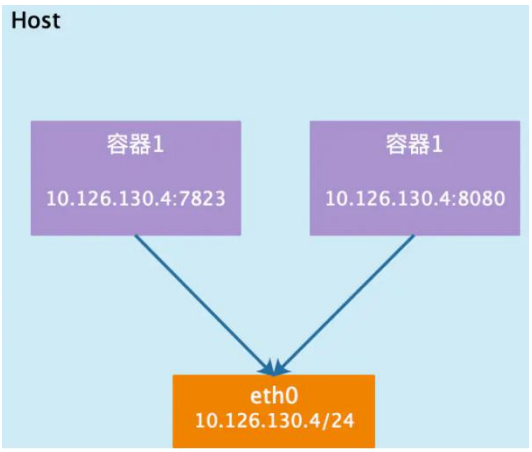
Docker 网络模式	配置	说明
host 模式	--net=host	容器和宿主机共享 Network namespace。
container 模式	--net=container:NAME*or*ID	容器和另外一个容器共享 Network namespace。kubernetes 中的 pod 就是多个容器共享一个 Network namespace。
none 模式	--net=none	容器有独立的 Network namespace，但并没有对其进行任何网络设置，如分配 veth pair 和网桥连接，配置 IP 等。
bridge 模式	--net=bridge	（默认为该模式）

host 模式

如果启动容器的时候使用 host 模式，那么这个容器将不会获得一个独立的 Network Namespace，而是和宿主机共用一个 Network Namespace。容器将不会虚拟出自己的网卡，配置自己的 IP 等，而是使用宿主机的 IP 和端口。但是，容器的其他方面，如文件系统、进程列表等还是和宿主机隔离的。

使用 host 模式的容器可以直接使用宿主机的 IP 地址与外界通信，容器内部的服务端口也可以使用宿主机的端口，不需要进行 NAT，host 最大的优势就是网络性能比较好，但是 docker host 上已经使用的端口就不能再用了，网络的隔离性不好。

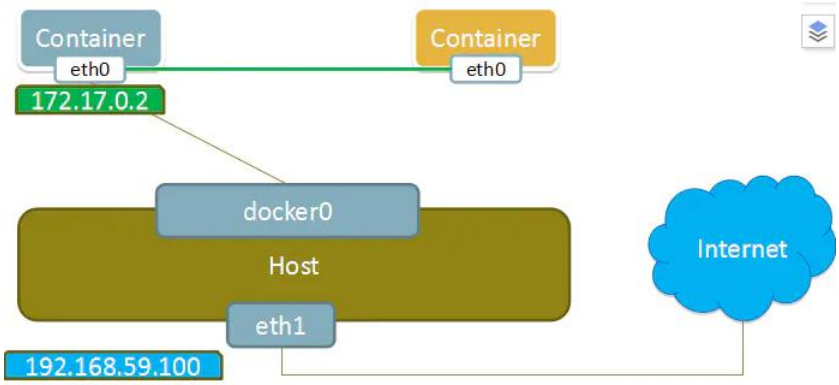
Host 模式如下图所示：



container 模式

这个模式指定新创建的容器和已经存在的一个容器共享一个 Network Namespace，而不是和宿主机共享。新创建的容器不会创建自己的网卡，配置自己的 IP，而是和一个指定的容器共享 IP、端口范围等。同样，两个容器除了网络方面，其他的如文件系统、进程列表等还是隔离的。两个容器的进程可以通过 lo 网卡设备通信。

Container 模式示意图：

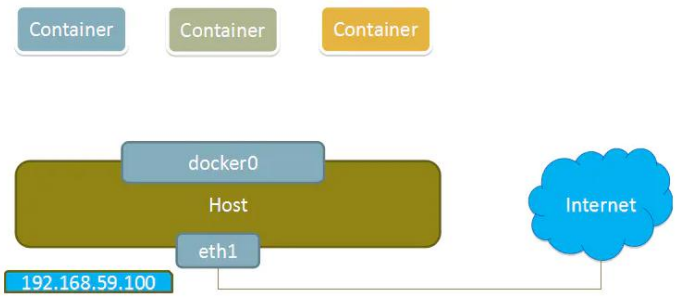


none 模式

使用 none 模式，Docker 容器拥有自己的 Network Namespace，但是，并不为 Docker 容器进行任何网络配置。也就是说，这个 Docker 容器没有网卡、IP、路由等信息。需要我们自己为 Docker 容器添加网卡、配置 IP 等。

这种网络模式下容器只有 lo 回环网络，没有其他网卡。none 模式可以在容器创建时通过 --network=none 来指定。这种类型的网络没有办法联网，封闭的网络能很好的保证容器的安全性。

None 模式示意图:

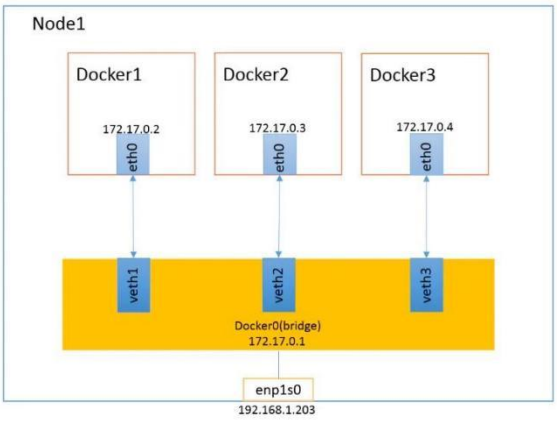


bridge 模式

当 Docker 进程启动时，会在主机上创建一个名为 docker0 的虚拟网桥，此主机上启动的 Docker 容器会连接到这个虚拟网桥上。虚拟网桥的工作方式和物理交换机类似，这样主机上的所有容器就通过交换机连在了一个二层网络中。

从 docker0 子网中分配一个 IP 给容器使用，并设置 docker0 的 IP 地址为容器的默认网关。在主机上创建一对虚拟网卡 veth pair 设备，Docker 将 veth pair 设备的一端放在新创建的容器中，并命名为 eth0（容器的网卡），另一端放在主机中，以 vethxxx 这样类似的名字命名，并将这个网络设备加入到 docker0 网桥中。可以通过 brctl show 命令查看。

bridge 模式是 docker 的默认网络模式，不写--net 参数，就是 bridge 模式。使用 docker run -p 时，docker 实际是在 iptables 做了 DNAT 规则，实现端口转发功能。可以使用 iptables -t nat -vnL 查看。



8.2.5、docker network create 创建网络以及启动容器指定网络

```
[root@localhost ~]# docker network create --help
Usage:  docker network create [OPTIONS] NETWORK
```

创建一个 mysqlNet

--driver bridge　配置网络类型 bridge 桥接

--subnet 192.168.1.0/24　　配置子网 建议每个网络的范围尽量小

--gateway 192.168.1.1　　配置网关

```
docker network create --driver bridge --subnet 192.168.1.0/24 --gateway 192.168.1.1 mysqlNet
```

```
[root@localhost ~]# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
fd4b5495e02e        bridge              bridge              local
c942dbad2529        host                host                local
```

```
632596fe7cc3    mysqlNet    bridge    local
3d2cedfba066    none        null      local
```

启动容器指定网络

我们启动容器的时候可以加上 --net 参数可以指定启动容器的时候使用的网络，如果不加表示默认使用 docker0 网络

--net bridge 表示使用 docker0 网络

```
docker run -tid --name centos01 centos /bin/bash

docker run -tid --name centos01 --net bridge centos /bin/bash
```

--net mysqlNet 表示使用我们自定义网络

```
docker run -tid --name centos04 --net mysqlNet centos /bin/bash

docker run -tid --name centos05 --net mysqlNet centos /bin/bash
```

使用主机名称可以 ping 通

```
[root@localhost ~]# docker exec -ti centos05 /bin/bash
[root@a73d5bf51968 /]# ping centos04
PING centos04 (192.168.0.2) 56(84) bytes of data.
64 bytes from centos04.mysqlNet (192.168.0.2): icmp_seq=1 ttl=64 time=0.127 ms
64 bytes from centos04.mysqlNet (192.168.0.2): icmp_seq=2 ttl=64 time=0.054 ms
64 bytes from centos04.mysqlNet (192.168.0.2): icmp_seq=3 ttl=64 time=0.051 ms
```

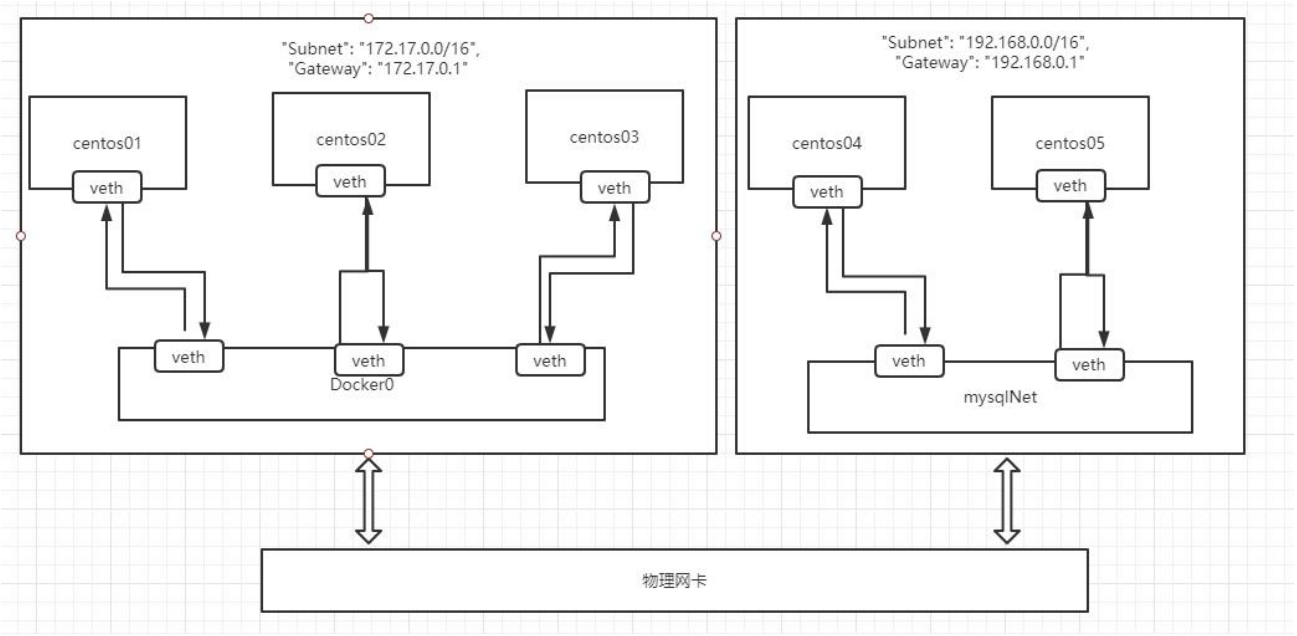
使用主机名称可以 ping 通

```
[root@localhost ~]# docker exec -ti centos05 /bin/bash
[root@a73d5bf51968 /]# ping centos04
PING centos04 (192.168.0.2) 56(84) bytes of data.
64 bytes from centos04.mysqlNet (192.168.0.2): icmp_seq=1 ttl=64 time=0.127 ms
64 bytes from centos04.mysqlNet (192.168.0.2): icmp_seq=2 ttl=64 time=0.054 ms
64 bytes from centos04.mysqlNet (192.168.0.2): icmp_seq=3 ttl=64 time=0.051 ms
```

不同网络的容器默认没法通信

```
[root@localhost ~]# docker exec centos05 ping 172.17.0.2
```

这样我们就把 centos04 和 centos05 加入了我们自定义的 mysqlNet 网络，这样的话 centos04 和 centos05 是互通的，但是 mysqlNet 网络和 docker0 网络默认是不互通的



8.2.6、docker network connect 实现不同网络之间的连通

如上图，我们想的是 centos01 可以 访问 mysqlNet 里面的 centos04 和 centos05，这个时候我们就需要使用 docker network connect 实现网络连通

```
[root@localhost ~]# docker network connect --help

Usage:  docker network connect [OPTIONS] NETWORK CONTAINER
```

```
docker network connect mysqlNet centos01
```

```
[root@localhost ~]# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
fd4b5495e02e        bridge              bridge              local
c942dbad2529        host                host                local
```

```
632596fe7cc3     mysqlNet     bridge       local
3d2cedfba066     none         null         local
```

```
[root@localhost ~]# docker network inspect mysqlNet
[
    {
        "Name": "mysqlNet",
        "Id": "632596fe7cc3eb080e916a50775acb9418e9798450ad3773fbfd7ff079bdf962",
        "Created": "2021-08-11T12:52:28.060697345+08:00",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "192.168.0.0/16",
                    "Gateway": "192.168.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {
            "4b602f76e865d71e2184c356f87bc9ab22a1b6c54cc945efd1463d1fcd105266": {
                "Name": "centos01",
                "EndpointID": "5ea7d4f81d96876b31f2e75d3925915071bbd5227dccdb2732
c024a2a2ac7605",
                "MacAddress": "02:42:c0:a8:00:04",
                "IPv4Address": "192.168.0.4/16",
                "IPv6Address": ""
            },
            "a73d5bf519685da6b52e94c79bb0b1ba980821159db77c93d2fa5b86ef114c31": {
                "Name": "centos05",
```

```
                "EndpointID": "682971c46967c7e4e4595f431f3e8a0d918a8491405ecf4f44ab54b45266e564",
                "MacAddress": "02:42:c0:a8:00:03",
                "IPv4Address": "192.168.0.3/16",
                "IPv6Address": ""
            },
            "e7793a41595627d25cf792def2ab8189fe966c966da5fd8b99a456929288baba": {
                "Name": "centos04",
                "EndpointID": "357c24e4d12d000d564db99099e4eb8dbcc10d8fd859fcc575c3fb8b2bd49092",
                "MacAddress": "02:42:c0:a8:00:02",
                "IPv4Address": "192.168.0.2/16",
                "IPv6Address": ""
            }
        },
        "Options": {},
        "Labels": {}
    }
]
```


```
[root@localhost ~]# docker exec centos01 ping centos05
PING centos05 (192.168.0.3) 56(84) bytes of data.
64 bytes from centos05.mysqlNet (192.168.0.3): icmp_seq=1 ttl=64 time=0.105 ms
64 bytes from centos05.mysqlNet (192.168.0.3): icmp_seq=2 ttl=64 time=0.043 ms
64 bytes from centos05.mysqlNet (192.168.0.3): icmp_seq=3 ttl=64 time=0.043 ms
```